Supplyshed Scenario Exploration

J. Stephenson

2026-09-30

- Read in data

```
knitr::opts_chunk$set(echo = TRUE,
                      cache = TRUE)#,
                      #fig.width = 12,
                      #fig.height = 12)

library("tidyverse"); theme_set(theme_bw())
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
## Warning: package 'tibble' was built under R version 4.4.3
```

```
## Warning: package 'tidyr' was built under R version 4.4.3
```

```
## Warning: package 'readr' was built under R version 4.4.3
```

```
## Warning: package 'purrr' was built under R version 4.4.3
```

```
## Warning: package 'dplyr' was built under R version 4.4.3
```

```
## Warning: package 'stringr' was built under R version 4.4.3
```

```
## Warning: package 'forcats' was built under R version 4.4.3
```

```
## Warning: package 'lubridate' was built under R version 4.4.3
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✓ dplyr     1.2.1     ✓ readr     2.2.0
## ✓ forcats   1.0.1     ✓ stringr   1.6.0
## ✓ ggplot2   4.0.3     ✓ tibble    3.3.1
## ✓ lubridate 1.9.5     ✓ tidyr     1.3.2
## ✓ purrr     1.2.2
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
e errors
```

```
library("factoextra")
```

```
## Welcome to factoextra!
## Want to learn more? See two factoextra-related books at https://www.datanovia.com/library/pri
ncipal-component-methods
```

```
library("cluster")
```

```
## Warning: package 'cluster' was built under R version 4.4.3
```

```
library("ggrepel")
```

```
## Warning: package 'ggrepel' was built under R version 4.4.3
```

```
library("fmsb")
```

```
## Warning: package 'fmsb' was built under R version 4.4.3
```

```
library("stringr")
library("ggplot2")

options(scipen = 999, width = 200)

dir.create("fig", showWarnings = FALSE)
```

```
sessionInfo()
```

```
## R version 4.4.1 (2024-06-14 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26100)
##
## Matrix products: default
##
##
## locale:
## [1] LC_COLLATE=English_United States.utf8  LC_CTYPE=English_United States.utf8    LC_MONETARY
=English_United States.utf8 LC_NUMERIC=C                           LC_TIME=English_United State
s.utf8
##
## time zone: America/Chicago
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] fmsb_0.7.6       ggrepel_0.9.8    cluster_2.1.8.2  factoextra_2.2.0 lubridate_1.9.5  for
cats_1.0.1    stringr_1.6.0    dplyr_1.2.1      purrr_1.2.2      readr_2.2.0      tidyr_1.3.2
## [12] tibble_3.3.1     ggplot2_4.0.3    tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] gtable_0.3.6       jsonlite_2.0.0     compiler_4.4.1     Rcpp_1.1.1         tidyselect_
1.2.1   dichromat_2.0-1    jquerylib_0.1.4    scales_1.4.0       yaml_2.3.12        fastmap_1.2.
0
## [11] R6_2.6.1           generics_0.1.4     knitr_1.52         tzdb_0.5.0         bslib_0.12.0
pillar_1.11.1      RColorBrewer_1.1-3 rlang_1.2.0        stringi_1.8.7      cachem_1.1.0
## [21] xfun_0.57          sass_0.4.10        S7_0.2.2           otel_0.2.0         timechange_
0.4.0   cli_3.6.6          withr_3.0.3        magrittr_2.0.5     digest_0.6.39      grid_4.4.1
## [31] rstudioapi_0.19.0  hms_1.1.4          lifecycle_1.0.5    vctrs_0.7.3        evaluate_1.
0.5     glue_1.8.1         farver_2.1.2       rmarkdown_2.32     tools_4.4.1        pkgconfig_2.
0.3
## [41] htmltools_0.5.9
```

Look at bioyield and transportation costs

```
wsArea <- read_csv("../data/raw/HUC12_list.csv") %>%
  mutate(area_ha = areaacres *0.404686,
         HUC12 = str_pad(huc12, width = 12, pad = "0"),
         NO3_loadkg = SUM_wsNO3load_lbs*0.45359,
         sed_loadkg = SUM_Mean_sed_tons*0.907185) %>%
  select(area_ha, HUC12,NO3_loadkg,sed_loadkg)
```

```
## Rows: 48 Columns: 6
## ── Column specification ────────────────────────────────────────────────────────
──────────
## Delimiter: ","
## chr (1): name
## dbl (5): areaacres, supArea_ac, huc12, SUM_wsNO3load_lbs, SUM_Mean_sed_tons
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Read in data

```
knitr::opts_chunk$set(echo = TRUE,
                      cache = TRUE)#,
                      #fig.width = 12,
                      #fig.height = 12)

library("tidyverse"); theme_set(theme_bw())

options(scipen = 999, width = 200)

dir.create("fig", showWarnings = FALSE)
```

```
## Generate dataframe
## crop acres = all cropped acres within the drainage area of the practice.
CBS_BIOraw <- read_csv("../data/raw/CBS_practiceSummary_BIOv4.csv") %>%
  mutate(practice = "CBS",
         pID = FBndID,
         IWAP = 2,
         INRS = "Perennial crops",
         biomass = "prairie",
         adjOCost = OpportunityCost,
         DirectCost_CP = DirectCost,
         TotalCost_CP = DirectCost_CP + OpportunityCost,
         opArea_ha = opAcres*0.404686, ## convert ac to ha
         patch_ha = 0,
         lin_ha = opArea_ha,
         rye_ha = 0,
         cropArea_ha = SUM_cropAcres*0.404686, ## convert ac to ha
         DC_haWH = DirectCost/opArea_ha,
         DCcp_haWH = DirectCost_CP/opArea_ha,
         TC_haWH = TotalCost/opArea_ha,
         TCcp_haWH = TotalCost_CP/opArea_ha,
         bioYield_Mg = bioYield * 0.9072, ## convert tons to Mg
         cpNO3ldkg = SUM_cpNO3load*0.453592, ## convert lb to kg
         cpNO3ldRedkg = SUM_cpNO3LoadReduced*0.453592, ## convert lb to kg
         cpSEDldkg = SUM_cpSEDloadlbs*0.453592, ## convert lb to kg
         cpSEDldRedkg = SUM_cpSEDLoadReduced*907.18, ## convert US tons to kg
         DC_kgNO3 = DirectCost / cpNO3ldRedkg,
         DCcp_kgNO3 = DirectCost_CP / cpNO3ldRedkg,
         TC_kgNO3 = TotalCost / cpNO3ldRedkg,
         TCcp_kgNO3 = TotalCost_CP / cpNO3ldRedkg,
         DC_kgSED = DirectCost / cpSEDldRedkg,
         DCcp_kgSED = DirectCost_CP / cpSEDldRedkg,
         TC_kgSED = TotalCost / cpSEDldRedkg,
         TCcp_kgSED = TotalCost_CP / cpSEDldRedkg,
         DC_MgCO2 = DirectCost / CO2red,
         DCcp_MgCO2 = DirectCost_CP / CO2red,
         TC_MgCO2 = TotalCost / CO2red,
         TCcp_MgCO2 = TotalCost_CP / CO2red,
         DC_MgN2O = DirectCost / N2Ored,
         DCcp_MgN2O = DirectCost_CP / N2Ored,
         TC_MgN2O = TotalCost / N2Ored,
         TCcp_MgN2O = TotalCost_CP / N2Ored,
         DC_MgBio = DirectCost / bioYield_Mg,
         DCcp_MgBio = DirectCost_CP / bioYield_Mg,
         TC_MgBio = TotalCost / bioYield_Mg,
         TCcp_MgBio = TotalCost_CP / bioYield_Mg,
         DC_enPot = DirectCost / energyPot,
         DCcp_enPot = DirectCost_CP / energyPot,
         TC_enPot = TotalCost / energyPot,
         TCcp_enPot = TotalCost_CP / energyPot) %>%
  select(MLRA,radial,HUC12,FBndID,pID,practice,biomass,opArea_ha,patch_ha,
         lin_ha,rye_ha,cropArea_ha,IWAP,
         INRS,DirectCost,DirectCost_CP,OpportunityCost,adjOCost,TotalCost,
```

```
        TotalCost_CP,DC_haWH,DCcp_haWH,TC_haWH,TCcp_haWH,biomass,bioYield_Mg,
        energyPot,CO2red,N2Ored,cpNO3ldkg,cpNO3ldRedkg,cpSEDldkg,cpSEDldRedkg,
        DC_kgNO3,DCcp_kgNO3,TC_kgNO3,TCcp_kgNO3,DC_kgSED,DCcp_kgSED,TC_kgSED,
        TCcp_kgSED,DC_MgBio,DCcp_MgBio,TC_MgBio,TCcp_MgBio,DC_enPot,DCcp_enPot,
        TC_enPot,TCcp_enPot,DC_MgCO2,DCcp_MgCO2,TC_MgCO2,TCcp_MgCO2,DC_MgN2O,
        DCcp_MgN2O,TC_MgN2O,TCcp_MgN2O)
```

```
## Rows: 1884 Columns: 31
## ── Column specification ────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────────────────────
──────────
## Delimiter: ","
## chr  (2): FBndID, HUC12
## dbl (29): OBJECTID, Shape_Length, Shape_Area, opAcres, DirectCost, meanPIdx, OpportunityCost,
bioYield, energyPot, CO2red, N2Ored, SUM_cropAcres, SUM_pctWScropAcres, MEAN_Nrate, MEAN_NO3con
c, SUM_...
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
## crop acres = all cropped acres within the drainage area of the practice.
NRW_BIOraw <- read_csv("../data/raw/NRW_practiceSummary_BIOv4.csv") %>%
  mutate(practice = "NRW",
         pID = SiteID,
         IWAP = 0,
         INRS = "Treatment wetlands",
         biomass = "prairie",
         adjOCost = OpportunityCost,
         TotalCost_CP = DirectCost_CP + OpportunityCost,
         opArea_ha = opAcres*0.404686, # convert ac to ha
         patch_ha = 0,
         lin_ha = 0,
         rye_ha = 0,
         bioYield_Mg = bioYield * 0.9072, ## convert tons to Mg
         cropArea_ha = SUM_cropAcres*0.404686, # convert ac to ha
         DC_haWH = DirectCost/opArea_ha,
         DCcp_haWH = DirectCost_CP/opArea_ha,
         TC_haWH = TotalCost/opArea_ha,
         TCcp_haWH = TotalCost_CP/opArea_ha,
         cpNO3ldkg = SUM_cpNO3load*0.453592, ## convert lb to kg
         cpNO3ldRedkg = SUM_cpNO3LoadReduced*0.453592, ## convert lb to kg
         cpSEDldkg = SUM_cpSEDloadlbs*0.453592, ## convert lb to kg
         cpSEDldRedkg = SUM_cpSEDLoadReduced*907.18, ## convert US tons to kg
         DC_kgNO3 = DirectCost / cpNO3ldRedkg,
         DCcp_kgNO3 = DirectCost_CP / cpNO3ldRedkg,
         TC_kgNO3 = TotalCost / cpNO3ldRedkg,
         TCcp_kgNO3 = TotalCost_CP / cpNO3ldRedkg,
         DC_kgSED = DirectCost / cpSEDldRedkg,
         DCcp_kgSED = DirectCost_CP / cpSEDldRedkg,
         TC_kgSED = TotalCost / cpSEDldRedkg,
         TCcp_kgSED = TotalCost_CP / cpSEDldRedkg,
         DC_MgCO2 = DirectCost / CO2red,
         DCcp_MgCO2 = DirectCost_CP / CO2red,
         TC_MgCO2 = TotalCost / CO2red,
         TCcp_MgCO2 = TotalCost_CP / CO2red,
         DC_MgN2O = DirectCost / N2Ored,
         DCcp_MgN2O = DirectCost_CP / N2Ored,
         TC_MgN2O = TotalCost / N2Ored,
         TCcp_MgN2O = TotalCost_CP / N2Ored,
         DC_MgBio = DirectCost / bioYield_Mg,
         DCcp_MgBio = DirectCost_CP / bioYield_Mg,
         TC_MgBio = TotalCost / bioYield_Mg,
         TCcp_MgBio = TotalCost_CP / bioYield_Mg,
         DC_enPot = DirectCost / energyPot,
         DCcp_enPot = DirectCost_CP / energyPot,
         TC_enPot = TotalCost / energyPot,
         TCcp_enPot = TotalCost_CP / energyPot) %>%
  select(MLRA,radial,HUC12,FBndID,pID,practice,biomass,opArea_ha,patch_ha,
         lin_ha,rye_ha,cropArea_ha,IWAP,
         INRS,DirectCost,DirectCost_CP,OpportunityCost,adjOCost,TotalCost,
         TotalCost_CP,DC_haWH,DCcp_haWH,TC_haWH,TCcp_haWH,biomass,bioYield_Mg,
         energyPot,CO2red,N2Ored,cpNO3ldkg,cpNO3ldRedkg,cpSEDldkg,cpSEDldRedkg,
```

```
        DC_kgNO3,DCcp_kgNO3,TC_kgNO3,TCcp_kgNO3,DC_kgSED,DCcp_kgSED,TC_kgSED,
        TCcp_kgSED,DC_MgBio,DCcp_MgBio,TC_MgBio,TCcp_MgBio,DC_enPot,DCcp_enPot,
        TC_enPot,TCcp_enPot,DC_MgCO2,DCcp_MgCO2,TC_MgCO2,TCcp_MgCO2,DC_MgN2O,
        DCcp_MgN2O,TC_MgN2O,TCcp_MgN2O)
```

```
## Rows: 428 Columns: 35
## ── Column specification ────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────────────────────
──────────
## Delimiter: ","
## chr  (2): HUC12, FBndID
## dbl (33): OBJECTID, SiteID, MAX_BuffAreaAc, Shape_Length, Shape_Area, opAcres, DirectCost, Di
rectCost_CP, meanPIdx, OpportunityCost, bioYield, energyPot, CO2red, N2Ored, SUM_cropAcres, SUM_
pctWScr...
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
## crop acres = field acres
PCP_BIOraw <- read_csv("../data/raw/PCP_practiceSummary_BIOv4.csv") %>%
  mutate(practice = "PCP",
         pID = FBndID,
         IWAP = 1,
         INRS = "Perennial crops",
         biomass = "prairie",
         DirectCost_CP = DirectCost,
         adjOCost = OpportunityCost,
         opArea_ha = opAcres*0.404686, # convert ac to ha
         patch_ha = opArea_ha,
         lin_ha = 0,
         rye_ha = 0,
         TotalCost_CP = DirectCost_CP + OpportunityCost,
         bioYield_Mg = bioYield * 0.9072, ## convert tons to Mg
         cropArea_ha = cropAcres*0.404686, # convert ac to ha
         DC_haWH = DirectCost/opArea_ha,
         DCcp_haWH = DirectCost_CP/opArea_ha,
         TC_haWH = TotalCost/opArea_ha,
         TCcp_haWH = TotalCost_CP/opArea_ha,
         cpNO3ldkg = cpNO3load*0.453592, ## convert lb to kg
         cpNO3ldRedkg = cpNO3LoadReduced*0.453592, ## convert lb to kg
         cpSEDldkg = cpSEDloadlbs*0.453592, ## convert lb to kg
         cpSEDldRedkg = cpSEDLoadReduced*907.18, ## convert US tons to kg
         DC_kgNO3 = DirectCost / cpNO3ldRedkg,
         DCcp_kgNO3 = DirectCost_CP / cpNO3ldRedkg,
         TC_kgNO3 = TotalCost / cpNO3ldRedkg,
         TCcp_kgNO3 = TotalCost_CP / cpNO3ldRedkg,
         DC_kgSED = DirectCost / cpSEDldRedkg,
         DCcp_kgSED = DirectCost_CP / cpSEDldRedkg,
         TC_kgSED = TotalCost / cpSEDldRedkg,
         TCcp_kgSED = TotalCost_CP / cpSEDldRedkg,
         DC_MgCO2 = DirectCost / CO2red,
         DCcp_MgCO2 = DirectCost_CP / CO2red,
         TC_MgCO2 = TotalCost / CO2red,
         TCcp_MgCO2 = TotalCost_CP / CO2red,
         DC_MgN2O = DirectCost / N2Ored,
         DCcp_MgN2O = DirectCost_CP / N2Ored,
         TC_MgN2O = TotalCost / N2Ored,
         TCcp_MgN2O = TotalCost_CP / N2Ored,
         DC_MgBio = DirectCost / bioYield_Mg,
         DCcp_MgBio = DirectCost_CP / bioYield_Mg,
         TC_MgBio = TotalCost / bioYield_Mg,
         TCcp_MgBio = TotalCost_CP / bioYield_Mg,
         DC_enPot = DirectCost / energyPot,
         DCcp_enPot = DirectCost_CP / energyPot,
         TC_enPot = TotalCost / energyPot,
         TCcp_enPot = TotalCost_CP / energyPot) %>%
  select(MLRA,radial,HUC12,FBndID,pID,practice,biomass,opArea_ha,patch_ha,
         lin_ha,rye_ha,cropArea_ha,IWAP,
         INRS,DirectCost,DirectCost_CP,OpportunityCost,adjOCost,TotalCost,
         TotalCost_CP,DC_haWH,DCcp_haWH,TC_haWH,TCcp_haWH,biomass,bioYield_Mg,
```

```
          energyPot,CO2red,N2Ored,cpNO3ldkg,cpNO3ldRedkg,cpSEDldkg,cpSEDldRedkg,
          DC_kgNO3,DCcp_kgNO3,TC_kgNO3,TCcp_kgNO3,DC_kgSED,DCcp_kgSED,TC_kgSED,
          TCcp_kgSED,DC_MgBio,DCcp_MgBio,TC_MgBio,TCcp_MgBio,DC_enPot,DCcp_enPot,
          TC_enPot,TCcp_enPot,DC_MgCO2,DCcp_MgCO2,TC_MgCO2,TCcp_MgCO2,DC_MgN2O,
          DCcp_MgN2O,TC_MgN2O,TCcp_MgN2O)
```

```
## Rows: 1108 Columns: 30
## ── Column specification ────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────────────────────
──────────
## Delimiter: ","
## chr  (2): FBndID, HUC12
## dbl (28): OBJECTID, OpportunityCost, bioYield, energyPot, CO2red, N2Ored, cropAcres, pctWScro
pAcres, cpNO3load, cpNO3LoadReduced, cpWSNO3LoadPct, cpSEDload, cpSEDloadlbs, cpSEDLoadReduced,
cpWSSED...
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
## crop acres = all cropped acres within the drainage area of the practice.
STB_BIOraw <- read_csv("../data/raw/STB_practiceSummary_BIOv4.csv") %>%
  mutate(practice = "STB",
         pID = riparianid,
         IWAP = 2,
         INRS = "Riparian buffers",
         biomass = "prairie",
         adjOCost = adjOCosts,
         TotalCost = adjOCost + DirectCost,
         TotalCost_CP = DirectCost_CP + adjOCost,
         opArea_ha = opAcres*0.404686, # convert ac to ha
         patch_ha = 0,
         lin_ha = opArea_ha,
         rye_ha = 0,
         bioYield_Mg = bioYield * 0.9072, ## convert tons to Mg
         cropArea_ha = SUM_cropAcres*0.404686, # convert ac to ha
         DC_haWH = DirectCost/opArea_ha,
         DCcp_haWH = DirectCost_CP/opArea_ha,
         TC_haWH = TotalCost/opArea_ha,
         TCcp_haWH = TotalCost_CP/opArea_ha,
         cpNO3ldkg = SUM_cpNO3load*0.453592, ## convert lb to kg
         cpNO3ldRedkg = SUM_cpNO3LoadReduced*0.453592, ## convert lb to kg
         cpSEDldkg = SUM_cpSEDloadlbs*0.453592, ## convert lb to kg
         cpSEDldRedkg = SUM_cpSEDLoadReduced*907.18, ## convert US tons to kg
         DC_kgNO3 = DirectCost / cpNO3ldRedkg,
         DCcp_kgNO3 = DirectCost_CP / cpNO3ldRedkg,
         TC_kgNO3 = TotalCost / cpNO3ldRedkg,
         TCcp_kgNO3 = TotalCost_CP / cpNO3ldRedkg,
         DC_kgSED = DirectCost / cpSEDldRedkg,
         DCcp_kgSED = DirectCost_CP / cpSEDldRedkg,
         TC_kgSED = TotalCost / cpSEDldRedkg,
         TCcp_kgSED = TotalCost_CP / cpSEDldRedkg,
         DC_MgCO2 = DirectCost / CO2red,
         DCcp_MgCO2 = DirectCost_CP / CO2red,
         TC_MgCO2 = TotalCost / CO2red,
         TCcp_MgCO2 = TotalCost_CP / CO2red,
         DC_MgN2O = DirectCost / N2Ored,
         DCcp_MgN2O = DirectCost_CP / N2Ored,
         TC_MgN2O = TotalCost / N2Ored,
         TCcp_MgN2O = TotalCost_CP / N2Ored,
         DC_MgBio = DirectCost / bioYield_Mg,
         DCcp_MgBio = DirectCost_CP / bioYield_Mg,
         TC_MgBio = TotalCost / bioYield_Mg,
         TCcp_MgBio = TotalCost_CP / bioYield_Mg,
         DC_enPot = DirectCost / energyPot,
         DCcp_enPot = DirectCost_CP / energyPot,
         TC_enPot = TotalCost / energyPot,
         TCcp_enPot = TotalCost_CP / energyPot) %>%
  select(MLRA,radial,HUC12,FBndID,pID,practice,biomass,opArea_ha,patch_ha,
         lin_ha,rye_ha,cropArea_ha,IWAP,
         INRS,DirectCost,DirectCost_CP,OpportunityCost,adjOCost,TotalCost,
         TotalCost_CP,DC_haWH,DCcp_haWH,TC_haWH,TCcp_haWH,biomass,bioYield_Mg,
```

```
        energyPot,CO2red,N2Ored,cpNO3ldkg,cpNO3ldRedkg,cpSEDldkg,cpSEDldRedkg,
        DC_kgNO3,DCcp_kgNO3,TC_kgNO3,TCcp_kgNO3,DC_kgSED,DCcp_kgSED,TC_kgSED,
        TCcp_kgSED,DC_MgBio,DCcp_MgBio,TC_MgBio,TCcp_MgBio,DC_enPot,DCcp_enPot,
        TC_enPot,TCcp_enPot,DC_MgCO2,DCcp_MgCO2,TC_MgCO2,TCcp_MgCO2,DC_MgN2O,
        DCcp_MgN2O,TC_MgN2O,TCcp_MgN2O)
```

```
## Rows: 1578 Columns: 37
## ── Column specification ────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────────────────────
──────────
## Delimiter: ","
## chr  (3): riparianid, HUC12, FBndID
## dbl (34): OBJECTID, Shape_Length, Shape_Area, bioAc, opAcres, DirectCost, DirectCost_CP, mean
PIdx, OpportunityCost, bioBuffWidth, adjOCosts, bioYield, energyPot, CO2red, N2Ored, SUM_cropAcr
es, SUM...
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
RIB_BIOraw <- read_csv("../data/raw/RIB_practiceSummary_BIOv4.csv") %>%
  mutate(practice = "RIB",
         pID = riparianid,
         IWAP = 2,
         INRS = "Riparian buffers",
         biomass = "prairie",
         adjOCost = adjOCosts,
         TotalCost = adjOCost + DirectCost,
         TotalCost_CP = DirectCost_CP + adjOCost,
         opArea_ha = opAcres*0.404686, # convert ac to ha
         patch_ha = 0,
         lin_ha = opArea_ha,
         rye_ha = 0,
         bioYield_Mg = bioYield * 0.9072, ## convert tons to Mg
         cropArea_ha = SUM_cropAcres*0.404686, # convert ac to ha
         DC_haWH = DirectCost/opArea_ha,
         DCcp_haWH = DirectCost_CP/opArea_ha,
         TC_haWH = TotalCost/opArea_ha,
         TCcp_haWH = TotalCost_CP/opArea_ha,
         cpNO3ldkg = SUM_cpNO3load*0.453592, ## convert lb to kg
         cpNO3ldRedkg = SUM_cpNO3LoadReduced*0.453592, ## convert lb to kg
         cpSEDldkg = SUM_cpSEDloadlbs*0.453592, ## convert lb to kg
         cpSEDldRedkg = SUM_cpSEDLoadReduced*907.18, ## convert US tons to kg
         DC_kgNO3 = DirectCost / cpNO3ldRedkg,
         DCcp_kgNO3 = DirectCost_CP / cpNO3ldRedkg,
         TC_kgNO3 = TotalCost / cpNO3ldRedkg,
         TCcp_kgNO3 = TotalCost_CP / cpNO3ldRedkg,
         DC_kgSED = DirectCost / cpSEDldRedkg,
         DCcp_kgSED = DirectCost_CP / cpSEDldRedkg,
         TC_kgSED = TotalCost / cpSEDldRedkg,
         TCcp_kgSED = TotalCost_CP / cpSEDldRedkg,
         DC_MgCO2 = DirectCost / CO2red,
         DCcp_MgCO2 = DirectCost_CP / CO2red,
         TC_MgCO2 = TotalCost / CO2red,
         TCcp_MgCO2 = TotalCost_CP / CO2red,
         DC_MgN2O = DirectCost / N2Ored,
         DCcp_MgN2O = DirectCost_CP / N2Ored,
         TC_MgN2O = TotalCost / N2Ored,
         TCcp_MgN2O = TotalCost_CP / N2Ored,
         DC_MgBio = DirectCost / bioYield_Mg,
         DCcp_MgBio = DirectCost_CP / bioYield_Mg,
         TC_MgBio = TotalCost / bioYield_Mg,
         TCcp_MgBio = TotalCost_CP / bioYield_Mg,
         DC_enPot = DirectCost / energyPot,
         DCcp_enPot = DirectCost_CP / energyPot,
         TC_enPot = TotalCost / energyPot,
         TCcp_enPot = TotalCost_CP / energyPot) %>%
  select(MLRA,radial,HUC12,FBndID,pID,practice,biomass,opArea_ha,patch_ha,
         lin_ha,rye_ha,cropArea_ha,IWAP,
         INRS,DirectCost,DirectCost_CP,OpportunityCost,adjOCost,TotalCost,
         TotalCost_CP,DC_haWH,DCcp_haWH,TC_haWH,TCcp_haWH,biomass,bioYield_Mg,
         energyPot,CO2red,N2Ored,cpNO3ldkg,cpNO3ldRedkg,cpSEDldkg,cpSEDldRedkg,
```

```
        DC_kgNO3,DCcp_kgNO3,TC_kgNO3,TCcp_kgNO3,DC_kgSED,DCcp_kgSED,TC_kgSED,
        TCcp_kgSED,DC_MgBio,DCcp_MgBio,TC_MgBio,TCcp_MgBio,DC_enPot,DCcp_enPot,
        TC_enPot,TCcp_enPot,DC_MgCO2,DCcp_MgCO2,TC_MgCO2,TCcp_MgCO2,DC_MgN2O,
        DCcp_MgN2O,TC_MgN2O,TCcp_MgN2O)
```

```
## Rows: 391 Columns: 37
## ── Column specification ────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────────────────────
──────────
## Delimiter: ","
## chr  (3): riparianid, HUC12, FBndID
## dbl (34): OBJECTID, Shape_Length, Shape_Area, bioAc, opAcres, DirectCost, DirectCost_CP, mean
PIdx, OpportunityCost, adjOCosts, bioBuffWidth, bioYield, energyPot, CO2red, N2Ored, SUM_cropAcr
es, SUM...
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
## crop acres = field acres
WCR_BIOraw <- read_csv("../data/raw/WCR_practiceSummary_BIOv4.csv") %>%
  mutate(practice = "WCR",
         pID = FBndID,
         IWAP = 0,
         INRS = "Rye cover crops",
         biomass = "rye",
         DirectCost_CP = DirectCost,
         adjOCost = OpportunityCost,
         opArea_ha = opAcres*0.404686, # convert ac to ha
         patch_ha = 0,
         lin_ha = 0,
         rye_ha = opArea_ha,
         bioYield_Mg = bioYield * 0.9072, ## convert tons to Mg
         cropArea_ha = cropAcres*0.404686, # convert ac to ha
         DC_haWH = 0,
         DCcp_haWH = 0,
         TC_haWH = 0,
         TCcp_haWH = 0,
         TotalCost = DirectCost,
         cpNO3ldkg = cpNO3load*0.453592, ## convert lb to kg
         cpNO3ldRedkg = cpNO3LoadReduced*0.453592, ## convert lb to kg
         cpSEDldkg = cpSEDloadlbs*0.453592, ## convert lb to kg
         cpSEDldRedkg = cpSEDLoadReduced*907.18, ## convert US tons to kg
         TotalCost_CP = TotalCost,
         DC_kgNO3 = DirectCost / cpNO3ldRedkg,
         DCcp_kgNO3 = DirectCost_CP / cpNO3ldRedkg,
         TC_kgNO3 = TotalCost / cpNO3ldRedkg,
         TCcp_kgNO3 = TotalCost_CP / cpNO3ldRedkg,
         DC_kgSED = DirectCost / cpSEDldRedkg,
         DCcp_kgSED = DirectCost_CP / cpSEDldRedkg,
         TC_kgSED = TotalCost / cpSEDldRedkg,
         TCcp_kgSED = TotalCost_CP / cpSEDldRedkg,
         DC_MgCO2 = DirectCost / CO2red,
         DCcp_MgCO2 = DirectCost_CP / CO2red,
         TC_MgCO2 = TotalCost / CO2red,
         TCcp_MgCO2 = TotalCost_CP / CO2red,
         DC_MgN2O = DirectCost / N2Ored,
         DCcp_MgN2O = DirectCost_CP / N2Ored,
         TC_MgN2O = TotalCost / N2Ored,
         TCcp_MgN2O = TotalCost_CP / N2Ored,
         DC_MgBio = DirectCost / bioYield_Mg,
         DCcp_MgBio = DirectCost_CP / bioYield_Mg,
         TC_MgBio = TotalCost / bioYield_Mg,
         TCcp_MgBio = TotalCost_CP / bioYield_Mg,
         DC_enPot = DirectCost / energyPot,
         DCcp_enPot = DirectCost_CP / energyPot,
         TC_enPot = TotalCost / energyPot,
         TCcp_enPot = TotalCost_CP / energyPot) %>%
  select(MLRA,radial,HUC12,FBndID,pID,practice,biomass,opArea_ha,patch_ha,
         lin_ha,rye_ha,cropArea_ha,IWAP,
         INRS,DirectCost,DirectCost_CP,OpportunityCost,adjOCost,TotalCost,
```

```
        TotalCost_CP,DC_haWH,DCcp_haWH,TC_haWH,TCcp_haWH,biomass,bioYield_Mg,
        energyPot,CO2red,N2Ored,cpNO3ldkg,cpNO3ldRedkg,cpSEDldkg,cpSEDldRedkg,
        DC_kgNO3,DCcp_kgNO3,TC_kgNO3,TCcp_kgNO3,DC_kgSED,DCcp_kgSED,TC_kgSED,
        TCcp_kgSED,DC_MgBio,DCcp_MgBio,TC_MgBio,TCcp_MgBio,DC_enPot,DCcp_enPot,
        TC_enPot,TCcp_enPot,DC_MgCO2,DCcp_MgCO2,TC_MgCO2,TCcp_MgCO2,DC_MgN2O,
        DCcp_MgN2O,TC_MgN2O,TCcp_MgN2O)
```

```
## Rows: 2830 Columns: 28
## ── Column specification ────────────────────────────────────────────────────────
──────────
## Delimiter: ","
## chr  (2): FBndID, HUC12
## dbl (26): OBJECTID, Shape_Length, Shape_Area, opAcres, DirectCost, OpportunityCost, bioYield,
energyPot, CO2red, N2Ored, cropAcres, pctWScropAcres, cpNO3load, cpNO3LoadReduced, cpWSNO3LoadPc
t, cpS...
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
practSumBIO_df <- rbind(CBS_BIOraw, NRW_BIOraw, PCP_BIOraw, STB_BIOraw, RIB_BIOraw, WCR_BIOraw)
practSumBIO <- within(practSumBIO_df, MLRA[MLRA == 108] <- "108c")
practSumBIO <- within(practSumBIO, MLRA[MLRA == 115] <- "115c")

fbSieversRoute <-  read_csv("../data/raw/fbSieversRoutes_v4.csv") %>%
  mutate(FBndID = trimws(sub("^([^-]+)-.*$", "\\1", Name),which="right"),
         rnd_dist = 2*Total_Kilometers,
         rnd_time_hr = (2*Total_TruckTravelTime)/60,
         #km_hr = Total_Kilometers / (Total_TruckTravelTime/60),
         fuel = 5.165*0.26417, # $ per gallon  --> $ per liter reference USDA AMS Transport Repo
rt 4/23/2026
         fuel_eff = 1/(5.5*0.425144), # mpg --> kpL --> Liters per km. This value was taken from
Gemini AI and should be fact-checked.
         fuel_cost = rnd_dist*fuel_eff*fuel,
         lbr_cost = rnd_time_hr*118, # 2026 ISU Custom Rate Survey. Haul round bales by truck or
trailer/ hour
         trans_cost = fuel_cost+lbr_cost
         ) %>%
  select(FBndID,Total_TruckTravelTime,Total_Kilometers,fuel_cost, lbr_cost, trans_cost)
```

```
## Rows: 5483 Columns: 6
## ── Column specification ────────────────────────────────────────────────────────
──────────
## Delimiter: ","
## chr (1): Name
## dbl (5): OBJECTID, FacilityID, Total_TruckTravelTime, Total_Kilometers, Shape_Length
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
## Generate dataframes
## crop acres = all cropped acres within the drainage area of the practice.
AvgYield <- read_csv("../data/raw/AvgYield012345678912.csv") %>%
  mutate(area_ha = Acres*0.404686, ## convert ac to ha
         subArea_ha = subfield_Acres*0.404686, ## convert bu per acre to kg per ha
         subfieldID = SubfieldID, ## convert ac to ha
         yrsCorn = Years_Corn,
         avgCorn_buha = AvgYield_Corn*0.405, ## convert bu per acre to kg per ha
         stdCorn_buha = StdYield_Corn*0.405, ## convert bu per acre to kg per ha
         cvCorn_per = Yield_CV_Corn*100,
         stabCorn = case_when(cvCorn_per < 5 ~ "Stable",
                              cvCorn_per < 10 ~ "Stable",
                              cvCorn_per < 30 ~ "Stable Low",
                              cvCorn_per > 30  ~ "Unstable"),
         yieldCornClass = case_when(avgCorn_buha < 81.4 | CountyName == "Cedar" ~ "Low", # Cedar
County 2015-2024 average = 201.4 bu per ac
                                    avgCorn_buha > 81.4 | CountyName == "Cedar" ~ "High",
                                    avgCorn_buha < 82.1 | CountyName == "Clint" ~ "Low", # Clint
on County 2015-2024 average = 202.7 bu per ac
                                    avgCorn_buha > 82.1 | CountyName == "Clint" ~ "High",
                                    avgCorn_buha < 79.4 | CountyName == "Musca" ~ "Low", # Musca
tine County 2015-2024 average = 196.1 bu per ac
                                    avgCorn_buha > 79.4 | CountyName == "Musca" ~ "High",
                                    avgCorn_buha < 82.8 | CountyName == "Scott" ~ "Low", # Scott
County 2015-2024 average = 204.4 bu per ac
                                    avgCorn_buha > 82.8 | CountyName == "Scott" ~ "High",
                                    avgCorn_buha < 80 | CountyName == "Jones" ~ "Low", # Jones C
ounty 2015-2024 average = 197.5 bu per ac
                                    avgCorn_buha > 80 | CountyName == "Jones" ~ "High"),
         yrsSoy = Years_Soy,
         avgSoy_buha = AvgYield_Soy*0.405, ## convert bu per acre to kg per ha
         stdSoy_buha = StdYield_Soy*0.405, ## convert bu per acre to kg per ha
         cvSoy_per = Yield_CV_Soy*100,
         stabSoy = case_when(cvSoy_per < 5 ~ "Stable High",
                             cvSoy_per < 10 ~ "Stable Low",
                             cvSoy_per <= 50  ~ "Unstable"),
         yieldSoyClass = case_when(avgSoy_buha < 24.6 | CountyName == "Cedar" ~ "Low", # Cedar C
ounty 2015-2024 average = 60.8 bu per ac
                                    avgSoy_buha > 24.6 | CountyName == "Cedar" ~ "High",
                                    avgSoy_buha < 24.3 | CountyName == "Clint" ~ "Low", # Clinto
n County 2015-2024 average = 60 bu per ac
                                    avgSoy_buha > 24.3 | CountyName == "Clint" ~ "High",
                                    avgSoy_buha < 24.2 | CountyName == "Musca" ~ "Low", # Muscat
ine County 2015-2024 average = 59.7 bu per ac
                                    avgSoy_buha > 24.2 | CountyName == "Musca" ~ "High",
                                    avgSoy_buha < 25.8 | CountyName == "Scott" ~ "Low", # Scott
County 2015-2024 average = 63.7 bu per ac
                                    avgSoy_buha > 25.8 | CountyName == "Scott" ~ "High",
                                    avgSoy_buha < 23.3 | CountyName == "Jones" ~ "Low", # Jones
County 2015-2024 average = 57.5 bu per ac
                                    avgSoy_buha > 23.3 | CountyName == "Jones" ~ "High"),
         AdjYield2015 = AdjYield2015*0.405,
```

```
         AdjYield2016 = AdjYield2016*0.405,
         AdjYield2017 = AdjYield2017*0.405,
         AdjYield2018 = AdjYield2018*0.405,
         AdjYield2019 = AdjYield2019*0.405,
         AdjYield2020 = AdjYield2020*0.405,
         AdjYield2021 = AdjYield2021*0.405,
         AdjYield2022 = AdjYield2022*0.405,
         AdjYield2023 = AdjYield2023*0.405,
         AdjYield2024 = AdjYield2024*0.405) %>%
  select(MLRA,HUC12,CountyName,FBndID,area_ha,subfieldID,subArea_ha,Sub_CSR2,
         TotalYears,NullYears,yrsCorn,avgCorn_buha,stdCorn_buha,cvCorn_per,stabCorn,
         yieldCornClass,yrsSoy,
         yieldSoyClass,avgSoy_buha,stdSoy_buha,cvSoy_per,stabSoy,,maj15,
         AdjYield2015,maj16,AdjYield2016,maj17,AdjYield2017,maj18,
         AdjYield2018,maj19,AdjYield2019,maj20,AdjYield2020,maj21,
         AdjYield2021,maj22,AdjYield2022,maj23,AdjYield2023,maj24,
         AdjYield2024)
```

```
## Rows: 100170 Columns: 46
## ── Column specification ────────────────────────────────────────────────────────
──────────
## Delimiter: ","
## chr  (7): FBndID, CountyName, SubfieldID, MLRA, HUC12, yieldCornClass, yieldSoyClass
## dbl (39): OBJECTID, Acres, isAG, FIPS, maj15, maj16, maj17, maj18, maj19, maj20, maj21, maj2
2, maj23, maj24, subfield_Acres, Sub_CSR2, Field_CSR2, AdjYield2015, AdjYield2016, AdjYield2017,
AdjYiel...
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
HUC12 <- read_csv("../data/raw/HUC12_list.csv") %>%
  mutate(HUC12_ha = areaacres*0.405,
         HUC12 = huc12) %>%
  select(HUC12, HUC12_ha, name)
```

```
## Rows: 48 Columns: 6
## ── Column specification ────────────────────────────────────────────────────────
──────────
## Delimiter: ","
## chr (1): name
## dbl (5): areaacres, supArea_ac, huc12, SUM_wsNO3load_lbs, SUM_Mean_sed_tons
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
yield <- merge(AvgYield, HUC12, by="HUC12")

AvgProfit <- read_csv("../data/raw/AvgProfit012345678912.csv") %>%
  mutate(area_ha = Acres*0.404686, ## convert ac to ha
         subArea_ha = subfield_Acres*0.404686, ## convert bu per acre to kg per ha
         subfieldID = SubfieldID,
         yrsCorn = rowSums(.[6:16] == 1, na.rm = TRUE),
         yrsSoy = rowSums(.[6:16] == 5, na.rm = TRUE),
         unprofit = case_when(AvgProfit <= 0 ~ "unprofitable",
                              AvgProfit >= 0 ~ "profitable")) %>%
  select(MLRA,HUC12,CountyName,FBndID,area_ha,subfieldID,subArea_ha,Sub_CSR2,
         yrsCorn,yrsSoy,AvgProfit,UnpYears,unprofit)
```

```
## Rows: 99626 Columns: 39
## ── Column specification ────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────────────────────────
──────────
## Delimiter: ","
## chr  (7): FBndID, CountyName, SubfieldID, AvgUnp, MajUnp, MLRA, HUC12
## dbl (32): OBJECTID, Acres, isAG, FIPS, maj15, maj16, maj17, maj18, maj19, maj20, maj21, maj2
2, maj23, maj24, subfield_Acres, Sub_CSR2, Field_CSR2, Profit2015, Profit2016, Profit2017, Profi
t2018, P...
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
HUC12 <- read_csv("../data/raw/HUC12_list.csv") %>%
  mutate(HUC12_ha = areaacres*0.405,
         HUC12 = huc12) %>%
  select(HUC12, HUC12_ha, name)
```

```
## Rows: 48 Columns: 6
## ── Column specification ────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────────────────────────
──────────
## Delimiter: ","
## chr (1): name
## dbl (5): areaacres, supArea_ac, huc12, SUM_wsNO3load_lbs, SUM_Mean_sed_tons
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
profit <- merge(AvgProfit, HUC12, by="HUC12")

profit_yield <- full_join(AvgProfit, AvgYield)
```

```
## Joining with `by = join_by(MLRA, HUC12, CountyName, FBndID, area_ha, subfieldID, subArea_ha,
Sub_CSR2, yrsCorn, yrsSoy)`
```

```
## How much is it going to cost to realize the INRS? How much to achieve wildlife goals?
envOutcome_CostSum <- practSumBIO %>%
  group_by(FBndID) %>%
  summarize(n       = n(),
            MLRA = max(MLRA),
            HUC12 = max(HUC12),
            radial = max(radial),
            IWAP = list(unique(IWAP)),
            practice = list(unique(practice)),

            ## Cost summary per field
            opArea_ha = sum(opArea_ha),
            sumDC = sum(DirectCost),
            sumOC = sum(adjOCost),
            sumTC = sum(TotalCost),
            sumDCcp = sum(DirectCost_CP),
            sumTCcp = sum(TotalCost_CP),

            ## Bio yield summary
            sum_BioMg = sum(bioYield_Mg),
            DC_BioMg = sum_BioMg / sumDC,
            DCcp_BioMg = sum_BioMg / sumDCcp,
            TC_BioMg = sum_BioMg / sumTC,
            TCcp_BioMg = sum_BioMg / sumTCcp,
            #BioMg_med = median(bioYield_Mg, na.rm = TRUE),
            #Q1_BioMg = quantile(bioYield_Mg, probs = c(0.25)),
            #Q3_BioMg = quantile(bioYield_Mg, probs = c(0.75)),
            #BioTC_Mg_med = median(TC_MgBio, na.rm = TRUE),
            #Q1TC_MgBio = quantile(TC_MgBio, probs = c(0.25)),
            #Q3TC_MgBio = quantile(TC_MgBio, probs = c(0.75)),
            #BioTCcp_Mg_med = median(TCcp_MgBio, na.rm = TRUE),
            #Q1TCcp_MgBio = quantile(TCcp_MgBio, probs = c(0.25)),
            #Q3TCcp_MgBio = quantile(TCcp_MgBio, probs = c(0.75)),

            ## Energy potential summary
            sumEnergyPot = sum(energyPot),
            DC_MMBtu = sumDC / sumEnergyPot,
            DCcp_MMBtu = sumDCcp / sumEnergyPot,
            TC_MMBtu = sumTC / sumEnergyPot,
            TCcp_MMBtu = sumTCcp / sumEnergyPot,
            #engPot_med = median(energyPot, na.rm = TRUE),
            #Q1_engPot = quantile(energyPot, probs = c(0.25)),
            #Q3_engPot = quantile(energyPot, probs = c(0.75)),
            #engPotTC_med = median(TC_enPot, na.rm = TRUE),
            #Q1TC_enPot = quantile(TC_enPot, probs = c(0.25)),
            #Q3TC_enPot = quantile(TC_enPot, probs = c(0.75)),
            #engPotTCcp_med = median(TCcp_enPot, na.rm = TRUE),
            #Q1TCcp_enPot = quantile(TCcp_enPot, probs = c(0.25)),
            #Q3TCcp_enPot = quantile(TCcp_enPot, probs = c(0.75)),

            ## Wildlife Summary
            sum_patch_ha = sum(patch_ha),
```

```
            sum_lin_ha = sum(lin_ha),
            sum_wh_ha = sum(lin_ha) + sum(patch_ha),
            sum_rye_ha = sum(rye_ha),
            DC_pha = sumDC / sum(patch_ha),
            DC_lha = sumDC / sum(lin_ha),
            DC_wha = sumDC / sum(lin_ha) + sum(patch_ha),
            DCcp_pha = sumDCcp / sum(patch_ha),
            DCcp_lha = sumDCcp / sum(lin_ha),
            DCcp_wha = sumDCcp / sum(lin_ha) + sum(patch_ha),
            TC_pha = sumTC / sum(patch_ha),
            TC_lha = sumTC / sum(lin_ha),
            TC_wha = sumTC / sum(lin_ha) + sum(patch_ha),
            TCcp_pha = sumTCcp / sum(patch_ha),
            TCcp_lha = sumTCcp / sum(lin_ha),
            TCcp_wha = sumTCcp / sum(lin_ha) + sum(patch_ha),
            #BioMg_med = median(bioYield_Mg, na.rm = TRUE),
            #Q1_BioMg = quantile(bioYield_Mg, probs = c(0.25)),
            #Q3_BioMg = quantile(bioYield_Mg, probs = c(0.75)),
            #WHTC_ha_med = median(TC_haWH, na.rm = TRUE),
            #Q1TC_MgBio = quantile(TC_MgBio, probs = c(0.25)),
            #Q3TC_MgBio = quantile(TC_MgBio, probs = c(0.75)),
            #WHTCcp_ha_med = median(TCcp_haWH, na.rm = TRUE),
            #Q1TCcp_MgBio = quantile(TCcp_MgBio, probs = c(0.25)),
            #Q3TCcp_MgBio = quantile(TCcp_MgBio, probs = c(0.75)),

            ## CO2 summary
            sumCO2 = sum(CO2red),
            DC_CO2 = sumDC / sumCO2,
            DCcp_CO2 = sumDCcp / sumCO2,
            TC_CO2 = sumTC / sumCO2,
            TCcp_CO2 = sumTCcp / sumCO2,
            #CO2red_med = median(CO2red, na.rm = TRUE),
            #Q1_CO2red = quantile(CO2red, probs = c(0.25)),
            #Q3_CO2red = quantile(CO2red, probs = c(0.75)),
            #CO2TC_Mg_med = median(TC_MgCO2, na.rm=TRUE),
            #Q1_CO2TC_Mg = quantile(TC_MgCO2, probs = c(0.25)),
            #Q3_CO2TC_Mg = quantile(TC_MgCO2, probs = c(0.75)),
            #CO2TCcp_Mg_med = median(TCcp_MgCO2, na.rm=TRUE),
            #Q1_CO2TCcp_Mg = quantile(TCcp_MgCO2, probs = c(0.25)),
            #Q3_CO2TCcp_Mg = quantile(TCcp_MgCO2, probs = c(0.75)),

            ## N2O summary
            sumN2Ored = sum(N2Ored),
            DC_N2Ored = sumDC / sumN2Ored,
            DCcp_N2Ored = sumDCcp / sumN2Ored,
            TC_N2Ored = sumTC / sumN2Ored,
            TCcp_N2Ored = sumTCcp / sumN2Ored,
            #N2Ored_med = median(N2Ored, na.rm=TRUE),
            #Q1_N2Ored = quantile(N2Ored, probs = c(0.25)),
            #Q3_N2Ored = quantile(N2Ored, probs = c(0.75)),
            #N2OTC_Mg_med = median(TC_MgN2O, na.rm=TRUE),
            #Q1_N2OTC_Mg = quantile(TC_MgN2O, probs = c(0.25)),
```

```
            #Q3_N2OTC_Mg = quantile(TC_MgN2O, probs = c(0.75)),
            #N2OTCcp_Mg_med = median(TCcp_MgN2O, na.rm=TRUE),
            #Q1_N2OTCcp_Mg = quantile(TCcp_MgN2O, probs = c(0.25)),
            #Q3_N2OTCcp_Mg = quantile(TCcp_MgN2O, probs = c(0.75)),

            # NO3 load summary
            sumNO3red = sum(cpNO3ldRedkg, na.rm = TRUE),
            DC_NO3red = sumDC / sumNO3red,
            DCcp_NO3red = sumDCcp / sumNO3red,
            TC_NO3red = sumTC / sumNO3red,
            TCcp_NO3red = sumTCcp / sumNO3red,
            #NO3red_med = median(cpNO3ldRedkg, na.rm = TRUE),
            #Q1_NO3red = quantile(cpNO3ldRedkg, probs = c(0.25), na.rm = TRUE),
            #Q3_NO3red = quantile(cpNO3ldRedkg, probs = c(0.75), na.rm = TRUE),
            #NO3TC_kg_med = median(TC_kgNO3, na.rm = TRUE),
            #Q1_NO3TC_kg = quantile(TC_kgNO3, probs = c(0.25), na.rm = TRUE),
            #Q3_NO3TC_kg = quantile(TC_kgNO3, probs = c(0.75), na.rm = TRUE),
            #NO3TCcp_kg_med = median(TCcp_kgNO3, na.rm = TRUE),
            #Q1_NO3TCcp_kg = quantile(TCcp_kgNO3, probs = c(0.25), na.rm = TRUE),
            #Q3_NO3TCcp_kg = quantile(TCcp_kgNO3, probs = c(0.75), na.rm = TRUE),

            ## Sediment load summary
            sumSEDred = sum(cpSEDldRedkg, na.rm = TRUE),
            DC_SEDred = sumDC / sumSEDred,
            DCcp_SEDred = sumDCcp / sumSEDred,
            TC_SEDred = sumTC / sumSEDred,
            TCcp_SEDred = sumTCcp / sumSEDred,
            #SEDred_med = median(cpSEDldRedkg, na.rm = TRUE),
            #Q1_SEDred = quantile(cpSEDldRedkg, probs = c(0.25), na.rm = TRUE),
            #Q3_SEDred = quantile(cpSEDldRedkg, probs = c(0.75), na.rm = TRUE),
            #SEDTC_kg_med = median(TC_kgSED, na.rm = TRUE),
            #Q1_SEDTC_kg = quantile(TC_kgSED, probs = c(0.25), na.rm = TRUE),
            #Q3_SEDTC_kg = quantile(TC_kgSED, probs = c(0.75), na.rm = TRUE),
            #SEDTCcp_kg_med = median(TCcp_kgSED, na.rm = TRUE),
            #Q1_SEDTCcp_kg = quantile(TCcp_kgSED, probs = c(0.25), na.rm = TRUE),
            #Q3_SEDTCcp_kg = quantile(TCcp_kgSED, probs = c(0.75), na.rm = TRUE),
            )

#write_csv(envOutcome_CostSum,"envOutcome_byField.csv")
```

```
travel_envOutcomes <- full_join(envOutcome_CostSum, fbSieversRoute)
```

```
## Joining with `by = join_by(FBndID)`
```

```
profit_trvl_env <- full_join(AvgProfit, travel_envOutcomes) %>%
  drop_na(n)
```

```
## Joining with `by = join_by(MLRA, HUC12, FBndID)`
```

```
travel_HUC12Sum <- travel_envOutcomes %>%
  #group_by(radial) %>%
  group_by(FBndID,HUC12) %>%
  drop_na(Total_Kilometers) %>%
  summarize(AOI = "supplyshed",
            opArea = sum(opArea_ha),
            n       = n(),
            ## Bio yield summary
            BioMg = sum(sum_BioMg),

            ## Energy potential summary
            EnergyPot = sum(sumEnergyPot),

            ## Cost summary per field
            DC = sum(sumDC),
            OC = sum(sumOC),
            TC = sum(sumTC),
            DCcp = sum(sumDCcp),
            TCcp = sum(sumTCcp),

            ## Wildlife habitat summary
            patch = sum(sum_patch_ha),
            lin = sum(sum_lin_ha),
            prairie = sum(sum_wh_ha),
            rye = sum(sum_rye_ha),

            ## CO2 summary
            CO2 = sum(sumCO2),

            ## N2O summary
            N2Ored = sum(sumN2Ored),

            # NO3 load summary
            NO3red = sum(sumNO3red, na.rm = TRUE),

            ## Sediment load summary
            SEDred = sum(sumSEDred, na.rm = TRUE),

            Ctruck = sum_BioMg*(2*Total_Kilometers)*0.265, # $ per field. trucking cost. 2026 IS
U Custom Rate survey. Transportation coefficient = "haul round bales by truck or trailer per bal
e per mile", assuming 0.68 Mg of hay in a bale (Hart etal. 2015).
            Ctruck_Mg = (2*Total_Kilometers)*0.265,
            Chand = sum_BioMg*4.7794, # $ per field. 2026 ISU Custom Rate survey. Loading/unload
ing = "moving large round bales to storage per bale", assuming 0.68 Mg of hay in a bale. Coeffic
ients are assumed to be fixed cost.
            Ctotal = Ctruck + Chand, # trucking + handling costs per field.
            rnd_dist = 2*Total_Kilometers,
            Ctotal_Mg = (Ctruck + Chand) / sum_BioMg, # transportation cost per Mg of biomass ha
rvested from a field
            Ctotal_Mg_km = Ctotal / sum_BioMg / rnd_dist,
            Cdeliv = Ctotal + sumDC, # cost per field for biomass
            Cdeliv_Mg = Cdeliv / sum_BioMg, # production + transportation cost per Mg delivered
```

```
from a field
            Cdeliv_Mg_km = Cdeliv / sum_BioMg / rnd_dist,
            Ccpdeliv = Ctotal + sumDCcp, # cost per field for biomass and structural components
            Ccpdeliv_Mg = Ccpdeliv / sum_BioMg,
            Ccpdeliv_Mg_km = Ccpdeliv / sum_BioMg / rnd_dist
            )
```

```
## `summarise()` has regrouped the output.
## ℹ Summaries were computed grouped by FBndID and HUC12.
## ℹ Output is grouped by FBndID.
## ℹ Use `summarise(.groups = "drop_last")` to silence this message.
## ℹ Use `summarise(.by = c(FBndID, HUC12))` for per-operation grouping (`?dplyr::dplyr_by`) ins
tead.
```

```
## need to run both exploratory_yields and exploratory_profits to get the profit_yield datafram
e.
profitHUC12 <- profit_yield %>%
  filter(unprofit == "unprofitable") %>%
  group_by(HUC12) %>%
  summarize(sum_UNPha = sum(subArea_ha)) %>%
  drop_na()

ysHUC12 <- profit_yield %>%
  #filter(stabCorn == "Unstable") %>%
  filter(stabCorn != "Stable Low") %>%
  group_by(HUC12) %>%
  summarize(sum_UNSTha = sum(subArea_ha)) #%>%
  #drop_na()
```

```
Ctravel_HUC12 <- travel_HUC12Sum %>%
  #group_by(radial) %>%
  group_by(HUC12) %>%
  summarize(sumOPha = sum(opArea),
            sumBioMg = sum(BioMg),

            ## Energy potential summary
            sumEnergyPot = sum(EnergyPot),
            sumDC = sum(DC),
            sumOC = sum(OC),
            sumTC = sum(TC),
            sumDCcp = sum(DCcp),
            sumTCcp = sum(TCcp),

            ## Wildlife habitat summary
            sumpatch = sum(patch),
            sumlin = sum(lin),
            sumprairie = sum(prairie),
            sumrye = sum(rye),

            ## CO2 summary
            sumCO2 = sum(CO2),

            ## N2O summary
            sumN2Ored = sum(N2Ored),

            # NO3 load summary
            sumNO3red = sum(NO3red, na.rm = TRUE),

            ## Sediment load summary
            sumSEDred = sum(SEDred, na.rm = TRUE),

            mean_dist = mean(rnd_dist, na.rm = TRUE),
            #sd_dist = sd(rnd_dist),
            #med_dist = median(rnd_dist, na.rm = TRUE),
            #Q1_dist = quantile(rnd_dist, probs = c(0.25), na.rm = TRUE),
            #Q3_dist = quantile(rnd_dist, probs = c(0.75), na.rm = TRUE),

            mean_Ctruck = mean(Ctruck, na.rm = TRUE),
            sum_Ctruck = sum(Ctruck),
            mean_Ctruck = mean(Ctruck_Mg, na.rm=TRUE),
            #sd_Ctruck = sd(Ctruck),
            #med_Ctruck = median(Ctruck_Mg, na.rm = TRUE),
            #Q1_Ctruck = quantile(Ctruck_Mg, probs = c(0.25), na.rm = TRUE),
            #Q3_Ctruck = quantile(Ctruck_Mg, probs = c(0.75), na.rm = TRUE),

            mean_Chand = mean(Chand, na.rm = TRUE),
            sum_Chand = sum(Chand, na.rm = TRUE),
            #sd_Chand = sd(Chand),
            #med_Chand = median(Chand, na.rm = TRUE),
            #Q1_Chand = quantile(Chand, probs = c(0.25), na.rm = TRUE),
            #Q3_Chand = quantile(Chand, probs = c(0.75), na.rm = TRUE),
```

```
            mean_Ctotal = mean(Ctotal, na.rm = TRUE),
            sum_Ctotal = sum(Ctotal, na.rm = TRUE),
            #sd_Ctotal = sd(Ctotal, na.rm = TRUE),
            #med_Ctotal = median(Ctotal, na.rm = TRUE),
            #Q1_Ctotal = quantile(Ctotal, probs = c(0.25), na.rm = TRUE),
            #Q3_Ctotal = quantile(Ctotal, probs = c(0.75), na.rm = TRUE),

            mean_Ctotal_Mg = mean(Ctotal_Mg, na.rm = TRUE),
            #sd_Ctotal_Mg = sd(Ctotal_Mg),
            #med_Ctotal_Mg = median(Ctotal_Mg, na.rm = TRUE),
            #Q1_Ctotal_Mg = quantile(Ctotal_Mg, probs = c(0.25), na.rm = TRUE),
            #Q3_Ctotal_Mg = quantile(Ctotal_Mg, probs = c(0.75), na.rm = TRUE),

            mean_Cdeliv = mean(Cdeliv, na.rm = TRUE),
            sum_Cdeliv = sum(Cdeliv, na.rm = TRUE),
            #sd_Cdeliv = sd(Cdeliv),
            #med_Cdeliv = median(Cdeliv, na.rm = TRUE),
            #Q1_Cdeliv = quantile(Cdeliv, probs = c(0.25), na.rm = TRUE),
            #Q3_Cdeliv = quantile(Cdeliv, probs = c(0.75), na.rm = TRUE),
            mean_Cdeliv_Mg = mean(Cdeliv_Mg, na.rm = TRUE),
            #sd_Cdeliv_Mg =sd(Cdeliv_Mg),
            #med_Cdeliv_Mg = median(Cdeliv_Mg, na.rm = TRUE),
            #Q1_Cdeliv_Mg = quantile(Cdeliv_Mg, probs = c(0.25), na.rm = TRUE),
            #Q3_Cdeliv_Mg = quantile(Cdeliv_Mg, probs = c(0.75), na.rm = TRUE)
  )

#write_csv(Ctravel_radial,"travelcosts_byRad.csv")
```

```
huc12_summary <- full_join(Ctravel_HUC12, wsArea)
```

```
## Joining with `by = join_by(HUC12)`
```

```
huc12_summary <- wsArea %>%
  full_join(Ctravel_HUC12) %>%
  full_join(ysHUC12) %>%
  full_join(profitHUC12) %>%
  drop_na(HUC12,sumOPha) %>%
  replace_na(list(sum_UNSTha = 0)) ## 7 HUC12 watersheds had 0 subfields designated as unstable
```

```
## Joining with `by = join_by(HUC12)`
## Joining with `by = join_by(HUC12)`
## Joining with `by = join_by(HUC12)`
```

```
huc12_df <- huc12_summary %>%
  mutate(
    # ------------------------------------------------------------
    # Landscape-scale metrics
    #
    # The denominator is the portion of each HUC12 within the
    # 32-km supplyshed (`area_ha`), so all outcomes and treatment
    # areas are expressed relative to the same spatial extent.
    # ------------------------------------------------------------
    per_treated = sumOPha / area_ha,
    per_patch = sumpatch / area_ha,
    per_lin = sumlin / area_ha,
    per_prairie = sumprairie / area_ha,
    per_rye = sumrye / area_ha,

    bio_Mg_ha = sumBioMg / area_ha,
    dccp_ha = sumDCcp / area_ha,
    tccp_ha = sumTCcp / area_ha,

    ghg_ha = (sumCO2 + sumN2Ored) / area_ha,
    no3_ha = sumNO3red / area_ha,
    sed_ha = sumSEDred / area_ha,
    no3_red = sumNO3red/NO3_loadkg,
    sed_red = sumSEDred/sed_loadkg,


    stabLow_ha = sum_UNSTha / area_ha,
    unprofit_ha = sum_UNPha / area_ha,
    oc_ha = sumOC / area_ha,

    # ------------------------------------------------------------
    # Treatment-level performance metrics
    #
    # These are calculated per hectare treated and are kept
    # separate from the landscape-scale classification.
    # ------------------------------------------------------------
    no3_per_treated_ha = sumNO3red / sumOPha,
    sed_per_treated_ha = sumSEDred / sumOPha,
    ghg_per_treated_ha = (sumCO2 + sumN2Ored) / sumOPha,
    bio_per_treated_ha = sumBioMg / sumOPha,
    oc_per_treated_ha = sumOC / sumOPha,
    dccp_per_treated_ha = sumDCcp / sumOPha,
    tccp_per_treated_ha = sumTCcp / sumOPha,

    # Treatment composition, not efficiency
    prairie_share_treated = sumprairie / sumOPha,
    rye_share_treated = sumrye / sumOPha
  ) %>%
  filter(sumOPha > 0)
```

```
# ============================================================
# MULTIVARIATE HUC12 PLANNING CLASSIFICATION
# ============================================================
#
# The classification identifies recurring combinations of:
#   * conservation/practice configuration,
#   * landscape-scale environmental outcomes,
#   * biomass production, and
#   * treatment extent.
#
# The resulting classes are interpreted as watershed planning
# archetypes within the study landscape. They are not rankings
# and should not be interpreted as universally optimal
# configurations.
#
# PRIMARY ANALYSIS
# ----------------
# K-means clustering is performed on standardized ORIGINAL
# variables.
#
# PCA is used as a diagnostic to:
#   1. describe multivariate structure,
#   2. quantify variance explained,
#   3. visualize variable relationships, and
#   4. provide a sensitivity analysis using PCA scores.
#
# OPPORTUNITY COST
# ----------------
# Opportunity cost is NOT used to define the clusters because
# it is strongly coupled to land-use transformation and prairie
# extent. It is calculated separately and used to characterize
# the economic implementation context of each planning class.
# ============================================================


# ------------------------------------------------------------
# 0. Prepare HUC12 dataset
# ------------------------------------------------------------

huc12_planning <- huc12_df


# ------------------------------------------------------------
# 1. Define PRIMARY classification variables
# ------------------------------------------------------------
#
# These variables represent distinct planning dimensions:
#
#   per_treated = extent of landscape transformation
#   per_patch   = prairie patch configuration
#   per_lin     = linear habitat configuration
#   per_rye     = cereal rye configuration
```

```
#   no3_ha      = landscape-scale nitrate reduction
#   sed_ha      = landscape-scale sediment reduction
#   ghg_ha      = landscape-scale GHG reduction
#   bio_Mg_ha   = landscape-scale biomass production
#
# Direct cost is retained as an implementation requirement.
#
# Opportunity cost is intentionally excluded from clustering.
#
# IMPORTANT:
# If you decide that direct cost should NOT define the
# archetypes, remove dccp_ha from this vector and from the
# sensitivity analysis below.
# ------------------------------------------------------------

primary_vars <- c(
  "per_treated",
  "per_patch",
  "per_lin",
  "per_rye",
  "no3_ha",
  "sed_ha",
  "ghg_ha",
  "bio_Mg_ha",
  "dccp_ha"
)

cluster_vars <- huc12_planning %>%
  select(all_of(primary_vars))

# ------------------------------------------------------------
# 2. Retain complete HUC12s
# ------------------------------------------------------------

complete_idx <- complete.cases(cluster_vars)

huc12_cluster <- huc12_planning[complete_idx, ]

cluster_complete <- cluster_vars[complete_idx, ]


# Report sample size explicitly
cat("\nNumber of HUC12s available for classification:",
    nrow(cluster_complete), "\n")
```

```
##
## Number of HUC12s available for classification: 47
```

```
cat("Number excluded because of incomplete classification data:",
    sum(!complete_idx), "\n")
```

```
## Number excluded because of incomplete classification data: 0
```

```
# ------------------------------------------------------------
# 3. Standardize variables
# ------------------------------------------------------------
#
# All variables are standardized to mean = 0 and SD = 1.
# This prevents variables with larger numerical units from
# dominating Euclidean distance.
# ------------------------------------------------------------

cluster_scaled <- scale(cluster_complete)

cluster_scaled_df <- as.data.frame(cluster_scaled)
```

```
# ============================================================
# PCA DIAGNOSTICS
# ============================================================

# ------------------------------------------------------------
# 4. Correlation matrix
# ------------------------------------------------------------
#
# This is descriptive. It is included to document covariance
# among classification variables, not to claim that the
# variables are statistically independent.
# ------------------------------------------------------------

cluster_cor <- cor(
  cluster_complete,
  use = "pairwise.complete.obs"
)

print(round(cluster_cor, 3))
```

```
##             per_treated per_patch per_lin per_rye no3_ha sed_ha ghg_ha bio_Mg_ha dccp_ha
## per_treated       1.000     0.633   0.721   0.956  0.957  0.887  0.956     0.989   0.935
## per_patch         0.633     1.000   0.183   0.386  0.670  0.344  0.824     0.735   0.666
## per_lin           0.721     0.183   1.000   0.747  0.716  0.919  0.637     0.689   0.524
## per_rye           0.956     0.386   0.747   1.000  0.881  0.906  0.828     0.902   0.875
## no3_ha            0.957     0.670   0.716   0.881  1.000  0.867  0.948     0.964   0.933
## sed_ha            0.887     0.344   0.919   0.906  0.867  1.000  0.793     0.851   0.745
## ghg_ha            0.956     0.824   0.637   0.828  0.948  0.793  1.000     0.989   0.909
## bio_Mg_ha         0.989     0.735   0.689   0.902  0.964  0.851  0.989     1.000   0.932
## dccp_ha           0.935     0.666   0.524   0.875  0.933  0.745  0.909     0.932   1.000
```

```
write.csv(
  round(cluster_cor, 3),
  "../data/tidy/archetype_correlation_matrix.csv"
)


# ------------------------------------------------------------
# 5. PCA
# ------------------------------------------------------------
#
# PCA is performed on the standardized original variables.
#
# PCA is NOT the primary clustering space.
# It is used to:
#   - describe multivariate structure,
#   - quantify variance explained,
#   - examine variable contributions,
#   - visualize HUC12 relationships.
# ------------------------------------------------------------

pca_model <- prcomp(
  cluster_scaled,
  center = FALSE,
  scale. = FALSE
)

pca_summary <- summary(pca_model)

print(pca_summary)
```

```
## Importance of components:
##                           PC1    PC2     PC3     PC4     PC5     PC6      PC7      PC8
PC9
## Standard deviation     2.7230 1.0576 0.57909 0.28120 0.17839 0.14276 0.003966 0.001383 0.0000
6334
## Proportion of Variance 0.8239 0.1243 0.03726 0.00879 0.00354 0.00226 0.000000 0.000000 0.0000
0000
## Cumulative Proportion  0.8239 0.9482 0.98541 0.99420 0.99773 1.00000 1.000000 1.000000 1.0000
0000
```

```
# ------------------------------------------------------------
# 6. PCA variance explained
# ------------------------------------------------------------

pca_variance <- tibble(
  PC = paste0("PC", seq_along(pca_model$sdev)),
  eigenvalue = pca_model$sdev^2,
  variance_percent =
    100 * (pca_model$sdev^2 / sum(pca_model$sdev^2)),
  cumulative_percent =
    100 * cumsum(pca_model$sdev^2) /
    sum(pca_model$sdev^2)
)

print(pca_variance)
```

```
## # A tibble: 9 × 4
##   PC       eigenvalue variance_percent cumulative_percent
##   <chr>         <dbl>            <dbl>              <dbl>
## 1 PC1   7.41             82.4                        82.4
## 2 PC2   1.12             12.4                        94.8
## 3 PC3   0.335             3.73                       98.5
## 4 PC4   0.0791            0.879                      99.4
## 5 PC5   0.0318            0.354                      99.8
## 6 PC6   0.0204            0.226                     100.0
## 7 PC7   0.0000157         0.000175                  100.0
## 8 PC8   0.00000191        0.0000213                 100.0
## 9 PC9   0.00000000401     0.0000000446              100
```

```
write.csv(
  pca_variance,
  "../data/tidy/pca_variance_explained.csv",
  row.names = FALSE
)



# ------------------------------------------------------------
# 7. PCA loadings
# ------------------------------------------------------------

pca_loadings <- as.data.frame(
  pca_model$rotation
)

pca_loadings$Variable <- rownames(pca_loadings)

pca_loadings <- pca_loadings %>%
  relocate(Variable)

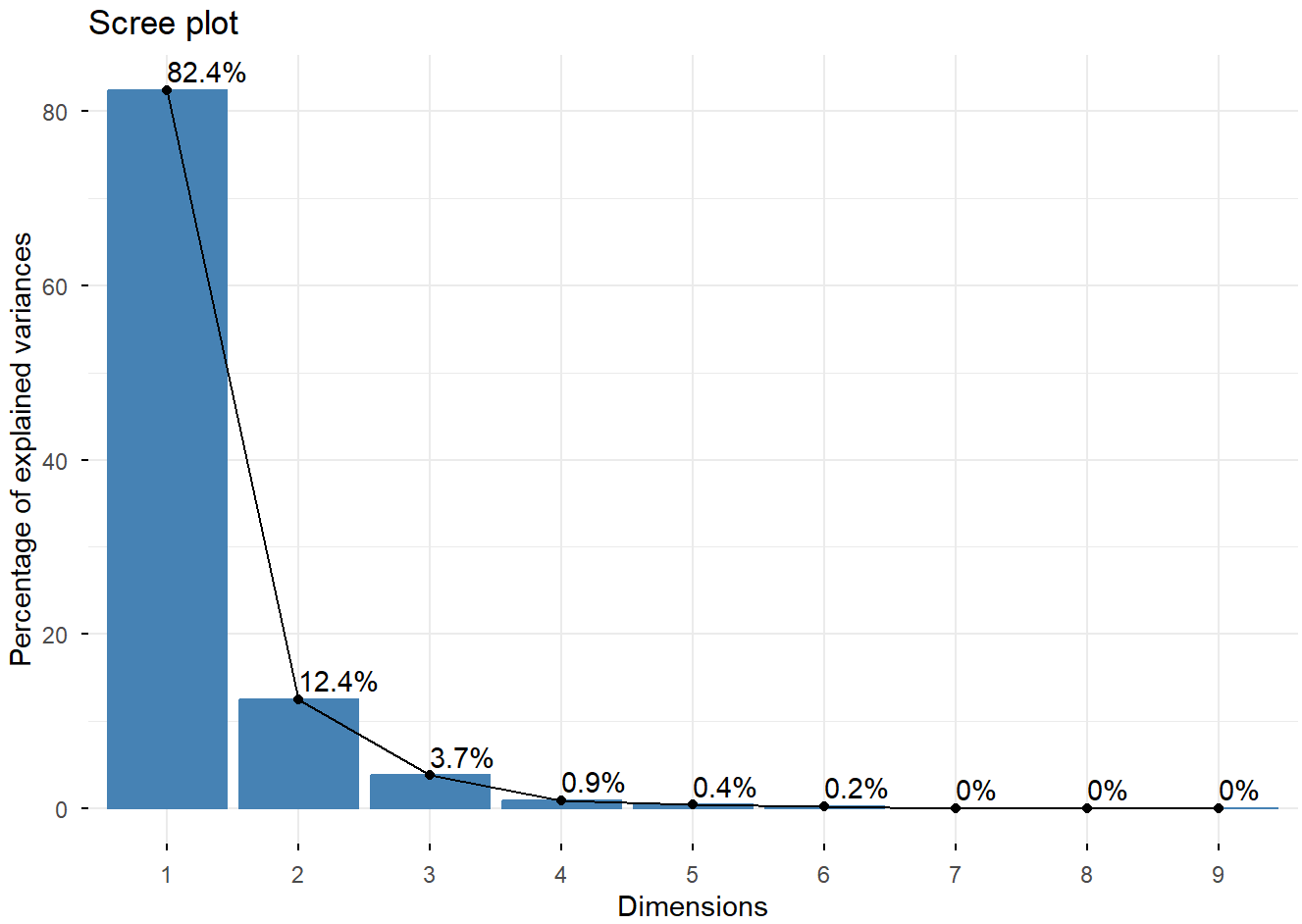
print(pca_loadings)
```

```
##                Variable       PC1          PC2          PC3         PC4         PC5         P
C6          PC7         PC8            PC9
## per_treated per_treated 0.3644207  0.002630832  0.152751121 -0.29912985  0.07604820 -0.097704
97 -0.338375393  0.09352057  0.78473275122
## per_patch     per_patch 0.2425163 -0.666688755 -0.441616193 -0.10835279 -0.08686412  0.095664
90 -0.355088716  0.30346076 -0.23467673069
## per_lin         per_lin 0.2775706  0.531332012 -0.553420236  0.12297398  0.53002254  0.072786
68 -0.118415294  0.11035626 -0.08259446878
## per_rye         per_rye 0.3411221  0.220871530  0.445687102 -0.43270116  0.04132586 -0.236196
92 -0.266245517  0.04498132 -0.56442659903
## no3_ha           no3_ha 0.3593036 -0.041042136  0.005396117  0.62207840 -0.19531420 -0.666039
57 -0.016704615 -0.01305166 -0.00028753894
## sed_ha           sed_ha 0.3317947  0.372190019 -0.147578878  0.01539103 -0.75879036  0.391898
97 -0.004175378 -0.00232418  0.00006927834
## ghg_ha           ghg_ha 0.3556495 -0.212138007 -0.166288783 -0.16594903  0.09875661  0.032194
85  0.168552619 -0.85640895 -0.02615693466
## bio_Mg_ha     bio_Mg_ha 0.3641900 -0.103801627 -0.009933397 -0.23085439  0.06864278 -0.060683
50  0.802868314  0.38631463  0.03110598906
## dccp_ha         dccp_ha 0.3415640 -0.168624030  0.476625648  0.47788792  0.27507170  0.564183
10 -0.034659006  0.04596321 -0.04550004402
```

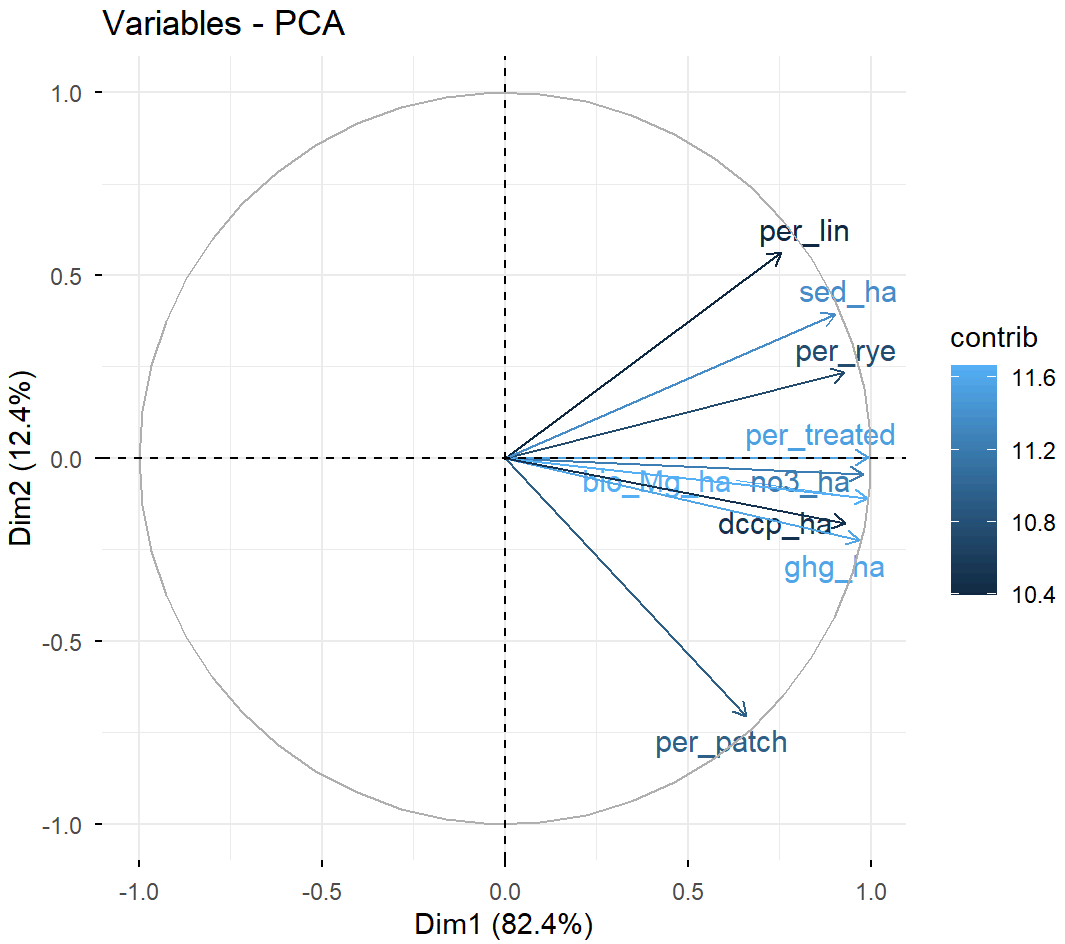
```
write.csv(
  pca_loadings,
  "../data/tidy/pca_loadings.csv",
  row.names = FALSE
)


# ------------------------------------------------------------
# 8. PCA diagnostic plots
# ------------------------------------------------------------

fviz_eig(
  pca_model,
  addlabels = TRUE
)
```




```
fviz_pca_var(
  pca_model,
  col.var = "contrib",
  repel = TRUE
)
```


Variables - PCA
1.0
0.5
0.0
-0.5
-1.0
Dim2 (12.4%)
-1.0
-0.5
0.0
0.5
1.0
Dim1 (82.4%)
per_lin
sed_ha
per_rye
per_treated
bio_Mg_ha
no3_ha
dccp_ha
ghg_ha
per_patch
contrib
11.6
11.2
10.8
10.4

```
# ============================================================
# NUMBER OF CLUSTERS
# ============================================================

# ------------------------------------------------------------
# 9. Evaluate candidate numbers of classes
# ------------------------------------------------------------
#
# We evaluate k = 2 through 6.
#
# WSS is an elbow diagnostic.
# Silhouette width evaluates within/between-cluster separation.
#
# Neither metric is used mechanically to select k.
# Cluster size and substantive interpretability are also
# considered.
# ------------------------------------------------------------

set.seed(123)

k_values <- 2:6

wss_results <- map_dfr(
  k_values,
  function(k) {

    km <- kmeans(
      cluster_scaled,
      centers = k,
      nstart = 100
    )

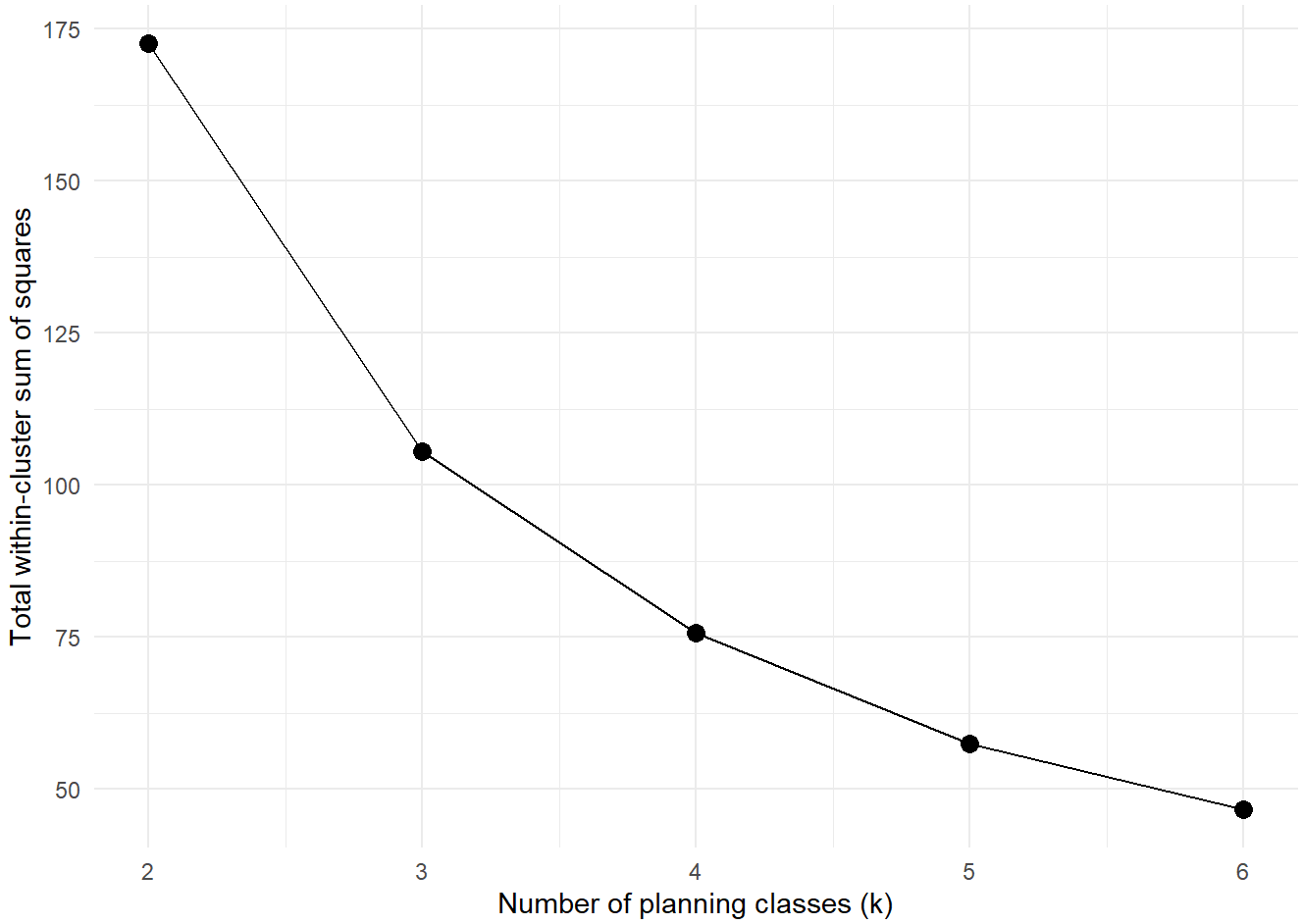
    tibble(
      k = k,
      tot_withinss = km$tot.withinss
    )
  }
)

print(wss_results)
```

```
## # A tibble: 5 × 2
##       k tot_withinss
##   <int>        <dbl>
## 1     2        173.
## 2     3        106.
## 3     4         75.6
## 4     5         57.4
## 5     6         46.6
```

```
# WSS plot

ggplot(
  wss_results,
  aes(x = k, y = tot_withinss)
) +
  geom_line() +
  geom_point(size = 3) +
  scale_x_continuous(breaks = k_values) +
  theme_minimal() +
  labs(
    x = "Number of planning classes (k)",
    y = "Total within-cluster sum of squares"
  )
```



```
# ------------------------------------------------------------
# 10. Silhouette diagnostics
# ------------------------------------------------------------

distance_matrix <- dist(cluster_scaled)

silhouette_results <- map_dfr(
  k_values,
  function(k) {

    set.seed(123)

    km <- kmeans(
      cluster_scaled,
      centers = k,
      nstart = 100
    )

    sil <- silhouette(
      km$cluster,
      distance_matrix
    )

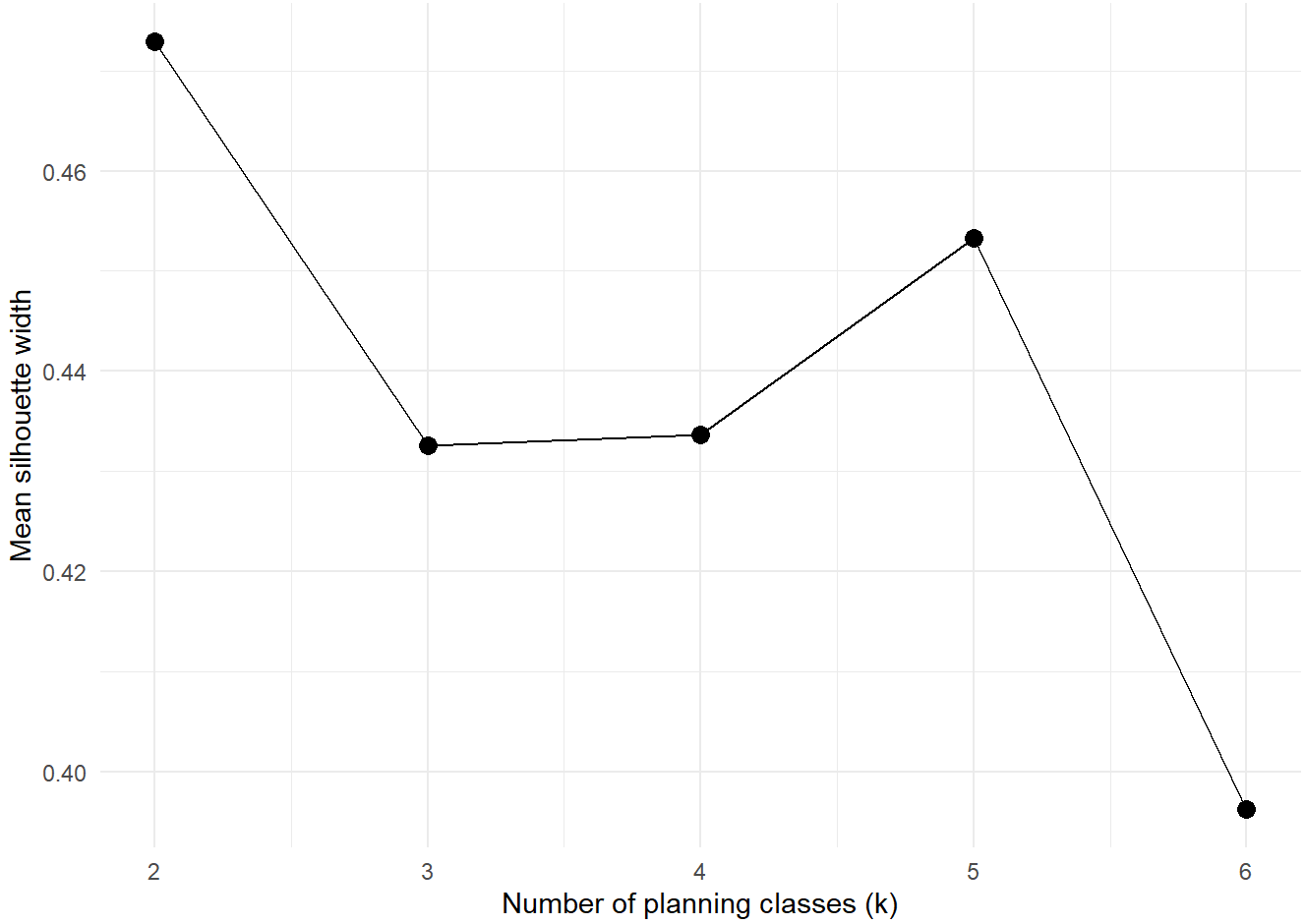
    tibble(
      k = k,
      mean_silhouette =
        mean(sil[, "sil_width"]),
      median_silhouette =
        median(sil[, "sil_width"]),
      min_silhouette =
        min(sil[, "sil_width"]),
      n_negative =
        sum(sil[, "sil_width"] < 0)
    )
  }
)

print(silhouette_results)
```

```
## # A tibble: 5 × 5
##       k mean_silhouette median_silhouette min_silhouette n_negative
##   <int>           <dbl>             <dbl>          <dbl>      <int>
## 1     2           0.473             0.508         0.134           0
## 2     3           0.433             0.414        -0.0423          1
## 3     4           0.434             0.419        -0.0457          2
## 4     5           0.453             0.456        -0.0981          2
## 5     6           0.396             0.371        -0.0421          3
```

```
write.csv(
  silhouette_results,
  "../data/tidy/silhouette_diagnostics.csv",
  row.names = FALSE
)


ggplot(
  silhouette_results,
  aes(x = k, y = mean_silhouette)
) +
  geom_line() +
  geom_point(size = 3) +
  scale_x_continuous(breaks = k_values) +
  theme_minimal() +
  labs(
    x = "Number of planning classes (k)",
    y = "Mean silhouette width"
  )
```



```
# ============================================================
# PRIMARY FIVE-CLASS SOLUTION
# ============================================================

# ------------------------------------------------------------
# 11. Fit primary k-means solution
# ------------------------------------------------------------
#
# Five classes are retained for the primary analysis because
# the five-class solution provides an additional meaningful
# distinction in practice configuration while maintaining
# comparable silhouette performance to the four-class solution.
#
# IMPORTANT:
# This is a substantive/model-selection decision, not a claim
# that five is universally optimal.
# ------------------------------------------------------------

k_primary <- 5

set.seed(123)

km_model <- kmeans(
  cluster_scaled,
  centers = k_primary,
  nstart = 100
)


# ------------------------------------------------------------
# 12. Add cluster assignment
# ------------------------------------------------------------

huc12_cluster <- huc12_cluster %>%
  mutate(
    cluster = factor(km_model$cluster)
  )


# ------------------------------------------------------------
# 13. Cluster sizes
# ------------------------------------------------------------

cluster_sizes <- huc12_cluster %>%
  count(cluster, name = "n") %>%
  mutate(
    percent = 100 * n / sum(n)
  )

print(cluster_sizes)
```

```
## # A tibble: 5 × 3
##   cluster     n percent
##   <fct>   <int>   <dbl>
## 1 1           5    10.6
## 2 2          10    21.3
## 3 3           8    17.0
## 4 4           6    12.8
## 5 5          18    38.3
```

```
write.csv(
  cluster_sizes,
  "../data/tidy/archetype_cluster_sizes.csv",
  row.names = FALSE
)



# ------------------------------------------------------------
# 14. Primary clustering diagnostics
# ------------------------------------------------------------

primary_silhouette <- silhouette(
  km_model$cluster,
  distance_matrix
)

primary_silhouette_summary <- tibble(
  k = k_primary,
  mean_silhouette =
    mean(primary_silhouette[, "sil_width"]),
  median_silhouette =
    median(primary_silhouette[, "sil_width"]),
  min_silhouette =
    min(primary_silhouette[, "sil_width"]),
  n_negative =
    sum(primary_silhouette[, "sil_width"] < 0)
)

print(primary_silhouette_summary)
```

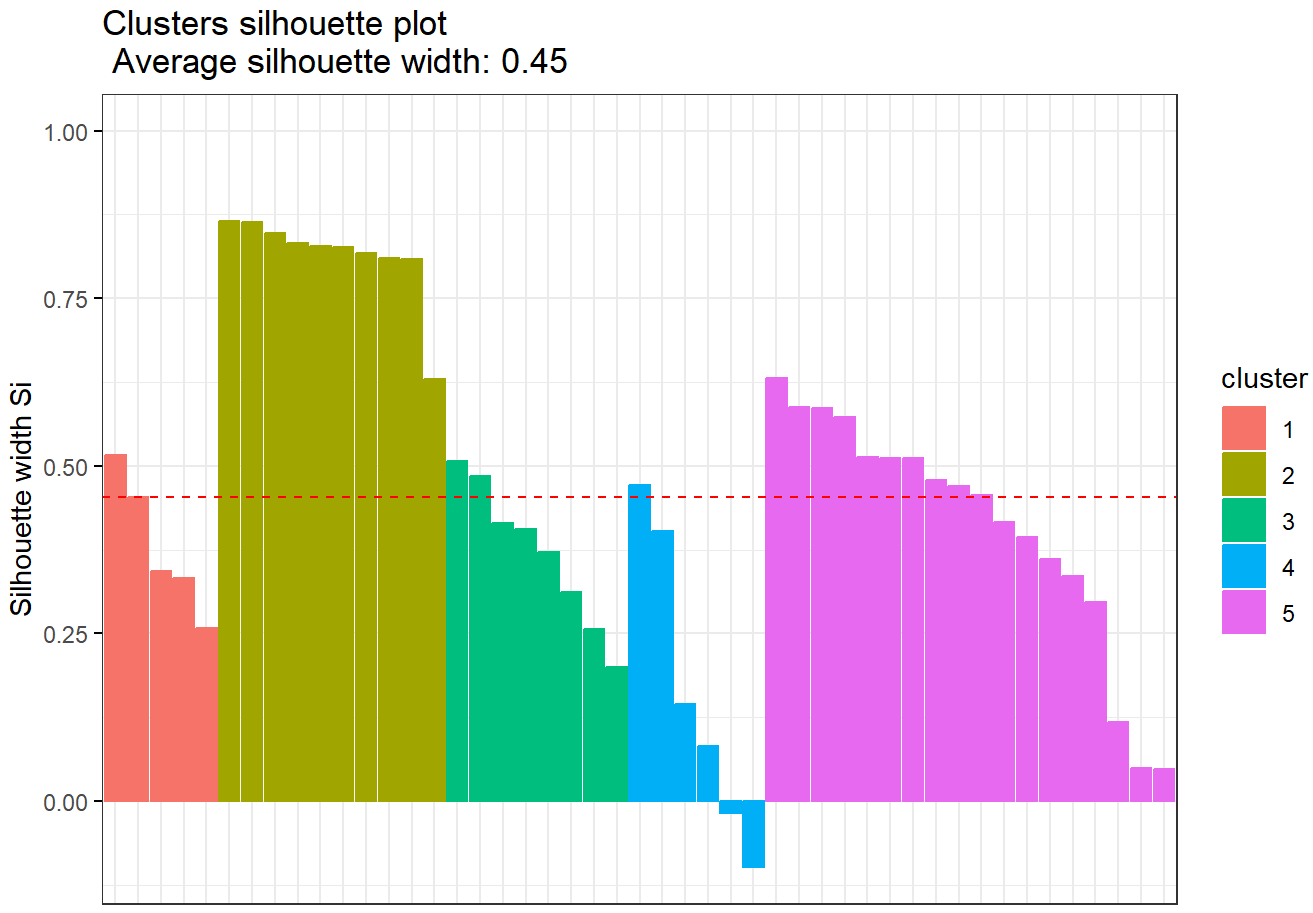
```
## # A tibble: 1 × 5
##       k mean_silhouette median_silhouette min_silhouette n_negative
##   <dbl>           <dbl>             <dbl>          <dbl>      <int>
## 1     5           0.453             0.456        -0.0981          2
```

```
write.csv(
  primary_silhouette_summary,
  "../data/tidy/primary_silhouette_summary.csv",
  row.names = FALSE
)


# Silhouette plot for the five-class solution

fviz_silhouette(
  primary_silhouette
)
```

```
##   cluster size ave.sil.width
## 1       1    5          0.38
## 2       2   10          0.81
## 3       3    8          0.37
## 4       4    6          0.16
## 5       5   18          0.41
```



```
# ============================================================
# CLUSTER STABILITY SENSITIVITY ANALYSIS
# ============================================================

# ------------------------------------------------------------
# 15. Repeated k-means stability analysis
# ------------------------------------------------------------
#
# K-means solutions can depend on starting values.
#
# We therefore repeat the five-class solution using many
# different random seeds and compare the resulting assignments
# to the primary solution using the Adjusted Rand Index (ARI).
#
# ARI:
#   1.0 = identical partition
#   ~0   = agreement approximately expected by chance
#
# This evaluates stability of the PARTITION, not statistical
# significance of individual variables.
# ------------------------------------------------------------

# Requires mclust
# install.packages("mclust") if necessary

library(mclust)
```

```
## Warning: package 'mclust' was built under R version 4.4.3
```

```
## Package 'mclust' version 6.1.2
## Type 'citation("mclust")' for citing this R package in publications.
```

```
##
## Attaching package: 'mclust'
```

```
## The following object is masked from 'package:dplyr':
##
##     count
```

```
## The following object is masked from 'package:purrr':
##
##     map
```

```
set.seed(123)

n_stability_runs <- 500

stability_results <- map_dfr(
  seq_len(n_stability_runs),
  function(i) {

    seed_i <- 1000 + i

    set.seed(seed_i)

    km_i <- kmeans(
      cluster_scaled,
      centers = k_primary,
      nstart = 25
    )

    ari_i <- adjustedRandIndex(
      km_model$cluster,
      km_i$cluster
    )

    tibble(
      run = i,
      seed = seed_i,
      ARI = ari_i
    )
  }
)

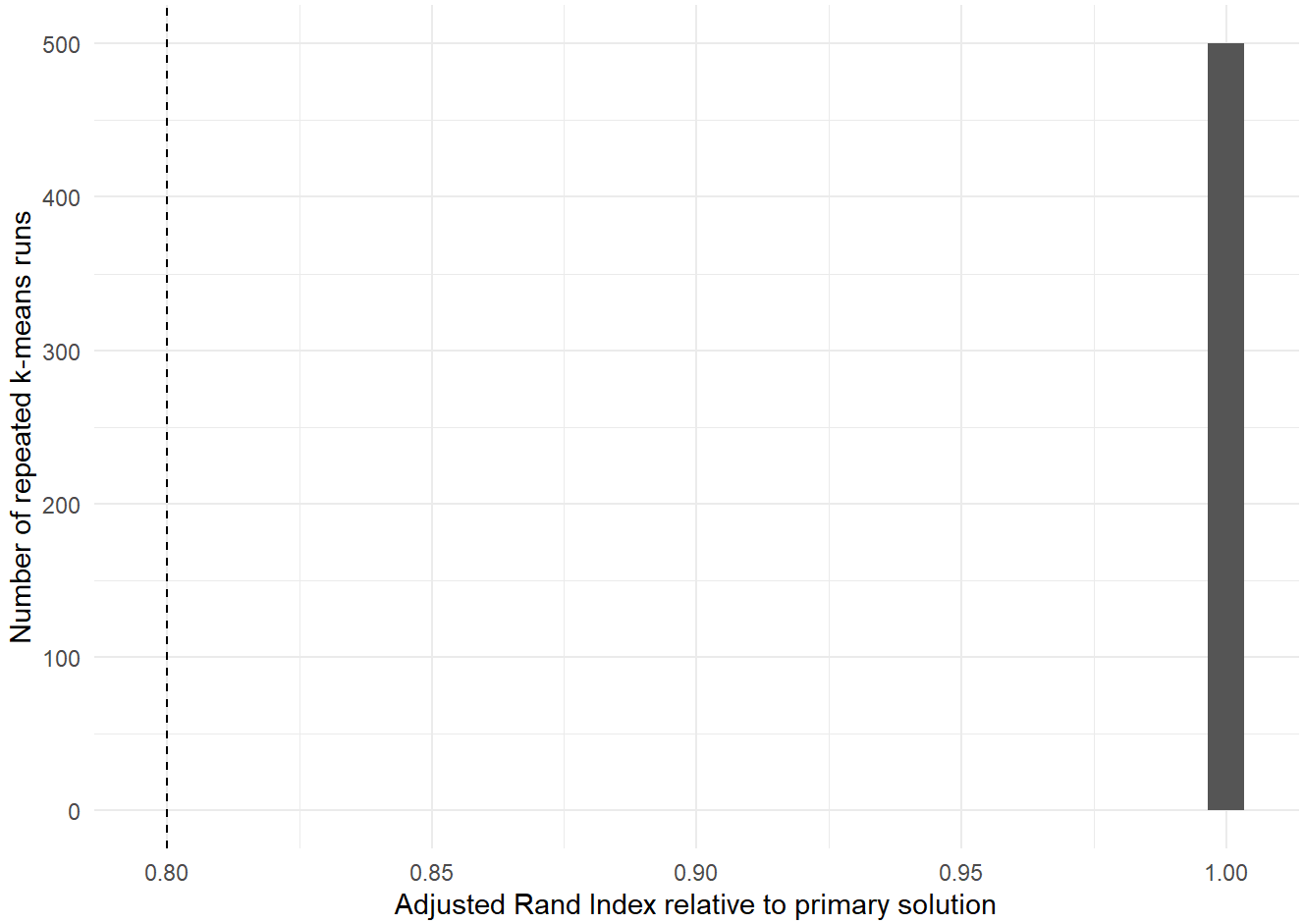
print(
  stability_results %>%
    summarise(
      mean_ARI = mean(ARI),
      median_ARI = median(ARI),
      min_ARI = min(ARI),
      max_ARI = max(ARI),
      sd_ARI = sd(ARI)
    )
)
```

```
## # A tibble: 1 × 5
##   mean_ARI median_ARI min_ARI max_ARI sd_ARI
##      <dbl>      <dbl>   <dbl>   <dbl>  <dbl>
## 1        1          1       1       1      0
```

```
write.csv(
  stability_results,
  "../data/tidy/archetype_stability_results.csv",
  row.names = FALSE
)


# Stability distribution

ggplot(
  stability_results,
  aes(x = ARI)
) +
  geom_histogram(bins = 30) +
  geom_vline(
    xintercept = 0.8,
    linetype = "dashed"
  ) +
  theme_minimal() +
  labs(
    x = "Adjusted Rand Index relative to primary solution",
    y = "Number of repeated k-means runs"
  )
```



```
# ============================================================
# SENSITIVITY TO NUMBER OF CLASSES
# ============================================================

# ------------------------------------------------------------
# 16. Compare cluster stability across k
# ------------------------------------------------------------
#
# This asks whether the selected number of classes is
# particularly unstable relative to nearby alternatives.
# ------------------------------------------------------------

set.seed(123)

k_stability <- 2:6

k_stability_results <- map_dfr(
  k_stability,
  function(k) {

    # Reference solution
    set.seed(123)

    reference <- kmeans(
      cluster_scaled,
      centers = k,
      nstart = 100
    )

    # Repeated solutions
    ari_values <- map_dbl(
      1:100,
      function(i) {

        set.seed(5000 + i)

        repeat_model <- kmeans(
          cluster_scaled,
          centers = k,
          nstart = 25
        )

        adjustedRandIndex(
          reference$cluster,
          repeat_model$cluster
        )
      }
    )

    tibble(
      k = k,
      mean_ARI = mean(ari_values),
```

```
      median_ARI = median(ari_values),
      min_ARI = min(ari_values),
      max_ARI = max(ari_values),
      sd_ARI = sd(ari_values)
    )
  }
)

print(k_stability_results)
```

```
## # A tibble: 5 × 6
##       k mean_ARI median_ARI min_ARI max_ARI sd_ARI
##   <int>    <dbl>      <dbl>   <dbl>   <dbl>  <dbl>
## 1     2    1              1   1           1 0
## 2     3    1              1   1           1 0
## 3     4    1              1   1           1 0
## 4     5    0.999          1   0.868       1 0.0132
## 5     6    1              1   1           1 0
```

```
write.csv(
  k_stability_results,
  "../data/tidy/k_stability_results.csv",
  row.names = FALSE
)


# ============================================================
# VARIABLE-SET SENSITIVITY ANALYSIS
# ============================================================

# ------------------------------------------------------------
# 17. Define alternative classification variable sets
# ------------------------------------------------------------
#
# The primary model contains strongly correlated variables.
# This is not necessarily inappropriate because the variables
# represent different planning dimensions, but it is important
# to determine whether the five-class solution depends heavily
# on particular variables.
#
# Sensitivity sets:
#
# A = Primary model
#
# B = Remove landscape transformation extent
#     (per_treated)
#
# C = Remove highly correlated transformation/biomass variables
#     (per_treated + bio_Mg_ha)
#
# D = Remove direct cost
#
# E = Practice/outcome configuration without direct cost and
#     biomass
#
# These are sensitivity analyses, not alternative "correct"
# models.
# ------------------------------------------------------------

variable_sets <- list(

  Primary = c(
    "per_treated",
    "per_patch",
    "per_lin",
    "per_rye",
    "no3_ha",
    "sed_ha",
    "ghg_ha",
    "bio_Mg_ha",
    "dccp_ha"
```

```
  ),

  No_Treatment_Extent = c(
    "per_patch",
    "per_lin",
    "per_rye",
    "no3_ha",
    "sed_ha",
    "ghg_ha",
    "bio_Mg_ha",
    "dccp_ha"
  ),

  No_Treatment_Extent_or_Biomass = c(
    "per_patch",
    "per_lin",
    "per_rye",
    "no3_ha",
    "sed_ha",
    "ghg_ha",
    "dccp_ha"
  ),

  No_Direct_Cost = c(
    "per_treated",
    "per_patch",
    "per_lin",
    "per_rye",
    "no3_ha",
    "sed_ha",
    "ghg_ha",
    "bio_Mg_ha"
  ),

  Configuration_and_Outcome = c(
    "per_patch",
    "per_lin",
    "per_rye",
    "no3_ha",
    "sed_ha",
    "ghg_ha"
  )
)


# ------------------------------------------------------------
# 18. Run sensitivity models
# ------------------------------------------------------------

sensitivity_models <- list()

sensitivity_summary <- map_dfr(
```

```
  names(variable_sets),
  function(model_name) {

    vars <- variable_sets[[model_name]]

    dat <- huc12_planning %>%
      select(all_of(vars)) %>%
      filter(complete.cases(.))

    scaled_dat <- scale(dat)

    set.seed(123)

    km <- kmeans(
      scaled_dat,
      centers = k_primary,
      nstart = 100
    )

    sil <- silhouette(
      km$cluster,
      dist(scaled_dat)
    )

    # Store model for later comparison
    sensitivity_models[[model_name]] <<- list(
      variables = vars,
      data = dat,
      scaled = scaled_dat,
      model = km
    )

    tibble(
      variable_set = model_name,
      n_variables = length(vars),
      n_HUC12 = nrow(dat),
      mean_silhouette =
        mean(sil[, "sil_width"]),
      median_silhouette =
        median(sil[, "sil_width"]),
      min_silhouette =
        min(sil[, "sil_width"]),
      n_negative =
        sum(sil[, "sil_width"] < 0)
    )
  }
)

print(sensitivity_summary)
```

```
## # A tibble: 5 × 7
##   variable_set                  n_variables n_HUC12 mean_silhouette median_silhouette min_si
lhouette n_negative
##   <chr>                               <int>   <int>           <dbl>             <dbl>
<dbl>      <int>
## 1 Primary                                 9      47           0.453             0.456
-0.0981          2
## 2 No_Treatment_Extent                     8      47           0.457             0.434
0.0341           0
## 3 No_Treatment_Extent_or_Biomass          7      47           0.452             0.442
0.0277           0
## 4 No_Direct_Cost                          8      47           0.465             0.466
-0.0822          1
## 5 Configuration_and_Outcome               6      47           0.454             0.477
0.0625           0
```

```
write.csv(
  sensitivity_summary,
  "../data/tidy/archetype_variable_sensitivity.csv",
  row.names = FALSE
)
```

```
# ============================================================
# 19. Compare alternative variable sets with PRIMARY solution
# ============================================================

variable_set_ARI <- map_dfr(
  names(sensitivity_models),
  function(model_name) {

    vars <- sensitivity_models[[model_name]]$variables

    # HUC12s with complete data for this variable set
    sensitivity_data <- huc12_planning %>%
      select(HUC12, all_of(vars)) %>%
      filter(complete.cases(.)) %>%
      arrange(HUC12)

    # Primary classifications for exactly the same HUC12s
    primary_data <- huc12_cluster %>%
      select(HUC12, primary_cluster = cluster) %>%
      filter(HUC12 %in% sensitivity_data$HUC12) %>%
      arrange(HUC12)

    # Refit sensitivity model using exactly these HUC12s
    x <- sensitivity_data %>%
      select(all_of(vars))

    x_scaled <- scale(x)

    set.seed(123)

    km_sens <- kmeans(
      x_scaled,
      centers = k_primary,
      nstart = 100
    )

    tibble(
      variable_set = model_name,
      n_HUC12_compared = nrow(sensitivity_data),
      ARI_vs_primary =
        adjustedRandIndex(
          primary_data$primary_cluster,
          km_sens$cluster
        )
    )
  }
)

print(variable_set_ARI)
```

```
## # A tibble: 5 × 3
##   variable_set                  n_HUC12_compared ARI_vs_primary
##   <chr>                                    <int>          <dbl>
## 1 Primary                                     47          1
## 2 No_Treatment_Extent                         47          0.868
## 3 No_Treatment_Extent_or_Biomass              47          0.868
## 4 No_Direct_Cost                              47          0.868
## 5 Configuration_and_Outcome                   47          0.799
```

```
write.csv(
  variable_set_ARI,
  "../data/tidy/archetype_variable_set_ARI.csv",
  row.names = FALSE
)
```

```
# ============================================================
# PCA-SCORE SENSITIVITY ANALYSIS
# ============================================================

# ------------------------------------------------------------
# 20. Cluster on PCA scores as a sensitivity analysis
# ------------------------------------------------------------
#
# This directly addresses an important methodological question:
#
# Does the classification change substantially if highly
# correlated original variables are first reduced to a smaller
# number of principal components?
#
# The PRIMARY classification remains based on the standardized
# original variables.
# ------------------------------------------------------------

# Determine number of PCs explaining at least 90% of variance

n_pc_90 <- which(
  pca_variance$cumulative_percent >= 90
)[1]

cat(
  "\nNumber of PCs explaining >=90% of variance:",
  n_pc_90,
  "\n"
)
```

```
##
## Number of PCs explaining >=90% of variance: 2
```

```
pca_scores_90 <- pca_model$x[, seq_len(n_pc_90), drop = FALSE]

set.seed(123)

km_pca <- kmeans(
  pca_scores_90,
  centers = k_primary,
  nstart = 100
)

pca_silhouette <- silhouette(
  km_pca$cluster,
  dist(pca_scores_90)
)

pca_sensitivity_summary <- tibble(
  model = paste0(
    "PCA scores (",
    n_pc_90,
    " PCs)"
  ),
  n_PC = n_pc_90,
  variance_explained =
    pca_variance$cumulative_percent[n_pc_90],
  mean_silhouette =
    mean(pca_silhouette[, "sil_width"]),
  median_silhouette =
    median(pca_silhouette[, "sil_width"]),
  min_silhouette =
    min(pca_silhouette[, "sil_width"])
)

print(pca_sensitivity_summary)
```

```
## # A tibble: 1 × 6
##   model               n_PC variance_explained mean_silhouette median_silhouette min_silhouett
e
##   <chr>              <int>              <dbl>           <dbl>             <dbl>          <dbl
>
## 1 PCA scores (2 PCs)     2               94.8           0.503             0.507         0.044
4
```

```
# Compare PCA solution with primary solution

pca_primary_ARI <- adjustedRandIndex(
  km_model$cluster,
  km_pca$cluster
)

cat(
  "\nAdjusted Rand Index: primary vs PCA-score solution =",
  round(pca_primary_ARI, 3),
  "\n"
)
```

```
##
## Adjusted Rand Index: primary vs PCA-score solution = 0.824
```

```
write.csv(
  pca_sensitivity_summary,
  "../data/tidy/pca_cluster_sensitivity.csv",
  row.names = FALSE
)
```

```
# ============================================================
# 21. Overall sensitivity summary
# ============================================================

overall_sensitivity <- bind_rows(

  sensitivity_summary %>%
    mutate(
      analysis_type = "Variable set"
    ) %>%
    select(
      analysis_type,
      model = variable_set,
      n_variables,
      n_HUC12,
      mean_silhouette,
      median_silhouette,
      min_silhouette,
      n_negative
    ),

  pca_sensitivity_summary %>%
    mutate(
      analysis_type = "PCA score sensitivity",
      n_variables = n_PC,
      n_HUC12 = nrow(pca_scores_90),
      n_negative =
        sum(pca_silhouette[, "sil_width"] < 0)
    ) %>%
    select(
      analysis_type,
      model,
      n_variables,
      n_HUC12,
      mean_silhouette,
      median_silhouette,
      min_silhouette,
      n_negative
    )
)

print(overall_sensitivity)
```

```
## # A tibble: 6 × 8
##   analysis_type         model                          n_variables n_HUC12 mean_silhouette me
dian_silhouette min_silhouette n_negative
##   <chr>                 <chr>                                <int>   <int>           <dbl>
<dbl>          <dbl>      <int>
## 1 Variable set          Primary                                  9      47           0.453
0.456        -0.0981          2
## 2 Variable set          No_Treatment_Extent                      8      47           0.457
0.434         0.0341          0
## 3 Variable set          No_Treatment_Extent_or_Biomass           7      47           0.452
0.442         0.0277          0
## 4 Variable set          No_Direct_Cost                           8      47           0.465
0.466        -0.0822          1
## 5 Variable set          Configuration_and_Outcome                6      47           0.454
0.477         0.0625          0
## 6 PCA score sensitivity PCA scores (2 PCs)                       2      47           0.503
0.507         0.0444          0
```

```
write.csv(
  overall_sensitivity,
  "../data/tidy/archetype_overall_sensitivity.csv",
  row.names = FALSE
)


# ============================================================
# 22. Visualize primary planning classes
# ============================================================

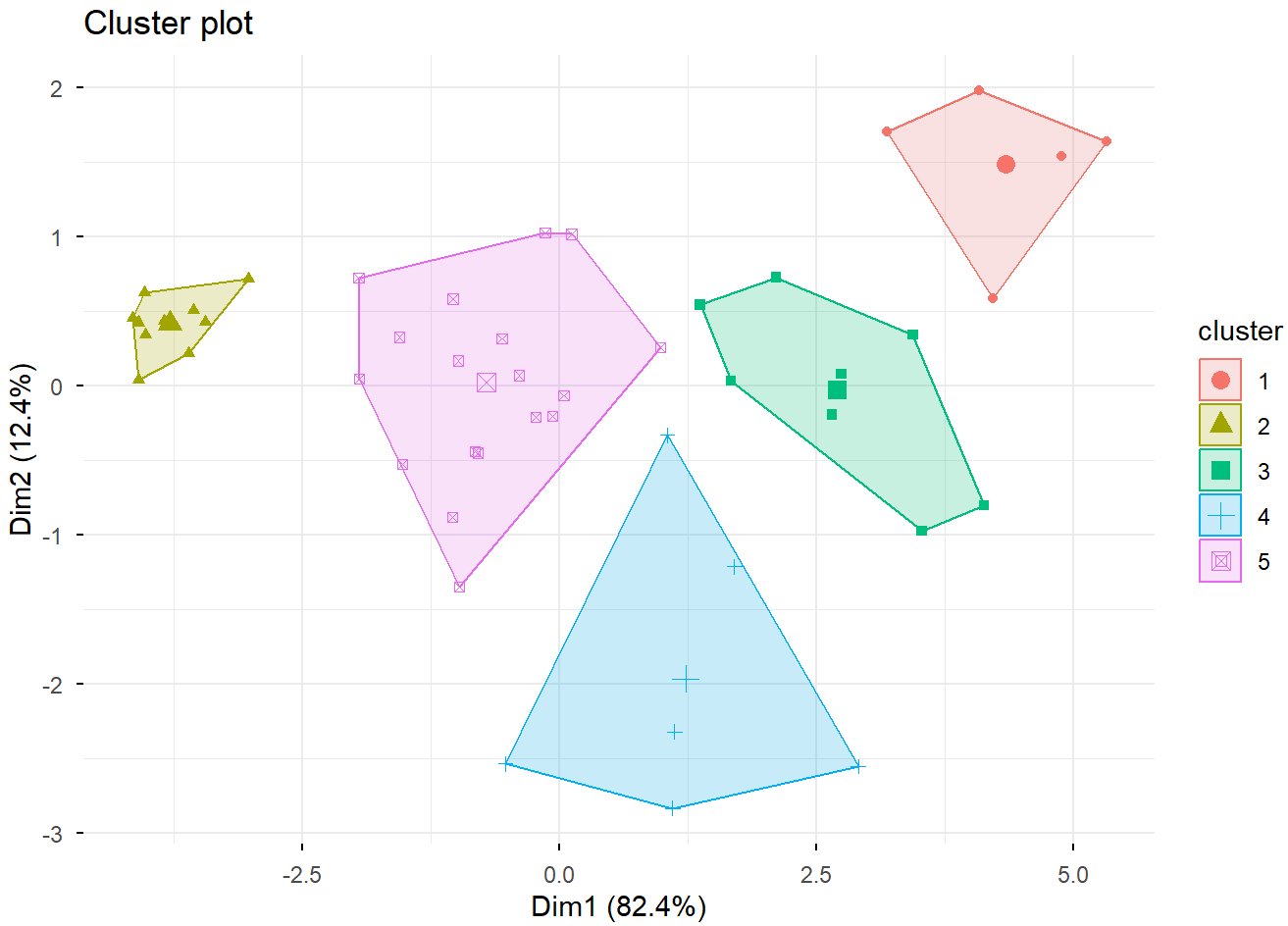
huc12_planning <- huc12_cluster %>%
  mutate(
    Planning_Class =
      paste0("Planning class ", cluster)
  )


# ------------------------------------------------------------
# Join classifications back to complete HUC12 dataset
# ------------------------------------------------------------

huc12_planning <- huc12_df %>%
  left_join(
    huc12_planning %>%
      select(
        HUC12,
        cluster,
        Planning_Class
      ),
    by = "HUC12"
  )


# ------------------------------------------------------------
# Cluster visualization
# ------------------------------------------------------------

fviz_cluster(
  km_model,
  data = cluster_scaled,
  geom = "point",
  ellipse.type = "convex",
  ggtheme = theme_minimal()
)
```


Cluster plot
Dim2 (12.4%)
Dim1 (82.4%)
2
1
0
-1
-2
-3
-2.5
0.0
2.5
5.0
cluster
1
2
3
4
5

```
# ============================================================
# TREATMENT-PERFORMANCE ANALYSIS
# ============================================================
#
# These metrics answer a different question from the landscape
# classification: how do outcomes and implementation intensity
# compare per hectare actually treated?
# ============================================================

# ============================================================
# 23. Summarize PRIMARY planning classes
# ============================================================

cluster_summary <- huc12_planning %>%
  filter(!is.na(cluster)) %>%
  group_by(cluster) %>%
  summarise(

    n = n(),

    # Landscape transformation
    mean_per_treated =
      mean(per_treated, na.rm = TRUE),

    # Practice configuration
    mean_per_patch =
      mean(per_patch, na.rm = TRUE),

    mean_per_lin =
      mean(per_lin, na.rm = TRUE),

    mean_per_rye =
      mean(per_rye, na.rm = TRUE),

    # Landscape-scale outcomes
    mean_no3_ha =
      mean(no3_ha, na.rm = TRUE),

    mean_sed_ha =
      mean(sed_ha, na.rm = TRUE),

    mean_ghg_ha =
      mean(ghg_ha, na.rm = TRUE),

    mean_bio_Mg_ha =
      mean(bio_Mg_ha, na.rm = TRUE),

    # Direct implementation cost
    mean_dccp_ha =
      mean(dccp_ha, na.rm = TRUE),

    # Opportunity cost -- CHARACTERIZATION ONLY
```

```
    mean_oc_ha =
      mean(oc_ha, na.rm = TRUE),

    # Total cost -- CHARACTERIZATION ONLY
    mean_tccp_ha =
      mean(tccp_ha, na.rm = TRUE),

    # Treatment-level outcomes
    mean_no3red =
      mean(no3_red, na.rm = TRUE),

    mean_sedred =
      mean(sed_red, na.rm = TRUE),

    .groups = "drop"
  )

print(cluster_summary)
```

```
## # A tibble: 5 × 15
##   cluster     n mean_per_treated mean_per_patch mean_per_lin mean_per_rye mean_no3_ha mean_se
d_ha mean_ghg_ha mean_bio_Mg_ha mean_dccp_ha mean_oc_ha mean_tccp_ha mean_no3red mean_sedred
##   <fct>   <int>            <dbl>          <dbl>        <dbl>        <dbl>       <dbl>       <
dbl>       <dbl>          <dbl>        <dbl>      <dbl>        <dbl>       <dbl>       <dbl>
## 1 1           5           0.477          0.0845      0.0544        0.333        16.5        3
076.      0.747          2.45         318.       303.         413.        0.714      72075.
## 2 2          10           0.0522         0.0152      0.00418       0.0323        1.28
229.      0.0910         0.281         34.7       28.6         45.8       0.200      14157.
## 3 3           8           0.414          0.0873      0.0238        0.294        14.3        1
870.      0.635          2.10         352.       292.         443.        0.607      61030.
## 4 4           6           0.307          0.149       0.0102        0.138        12.0
909.      0.642          1.83         285.       206.         396.        0.501      43811.
## 5 5          18           0.221          0.0601      0.0156        0.141         7.25
874.      0.373          1.17         174.       139.         223.        0.384      32268.
```

```
write.csv(
  cluster_summary,
  "../data/tidy/cluster_summary.csv",
  row.names = FALSE
)

# ------------------------------------------------------------
# 10. Treatment-performance distributions
# ------------------------------------------------------------
facet2_labels <- c(
  bio_per_treated_ha   = "atop(Biomass~Yield,(Mg~ha^{-1}))",
  no3_per_treated_ha      = "atop(NO[3]~Load~Red.,(Mg~ha^{-1}))",
  dccp_per_treated_ha     = "atop(Direct~Cost,(USD~ha^{-1}))",
  oc_per_treated_ha = "atop(Opportunity~Cost,(USD~ha^{-1}))",
  tccp_per_treated_ha = "atop(Total~Cost,(USD~ha^{-1}))",
  sed_per_treated_ha      = "atop(Sed.~Load~Red.,(Mg~ha^{-1}))",
  ghg_per_treated_ha      = "atop(GHG~Red.,(Mg~CO[2]*e~ha^{-1}))",
  per_treated = "Treated~('%')",
  prairie_share_treated   = "Prairie~('%')",
  rye_share_treated     = "Cereal~Rye~('%')"
)
huc12_planning %>%
  pivot_longer(
    cols = c(
      no3_per_treated_ha,
      sed_per_treated_ha,
      ghg_per_treated_ha,
      bio_per_treated_ha,
      dccp_per_treated_ha,
      rye_share_treated,
      prairie_share_treated,
      oc_per_treated_ha,

      #unprofit_ha
      #stabLow_ha,
    ),
    names_to = "Metric",
    values_to = "Value"
  ) %>%
  ggplot(
    aes(
      x = Planning_Class,
      y = Value,
      fill = Planning_Class
    )
  ) +
  geom_boxplot() +
  facet_wrap(~ factor(Metric, c("no3_per_treated_ha","sed_per_treated_ha","ghg_per_treated_h
a","bio_per_treated_ha","per_treated","rye_share_treated","prairie_share_treated","dccp_per_trea
ted_ha","oc_per_treated_ha","tccp_per_treated_ha")), scales = "free_y",ncol = 2,,
             strip.position = "left",
             labeller = as_labeller(facet2_labels, label_parsed)) +
```

```
  ylab(NULL) +
  theme_minimal() +
  theme(text = element_text(family = "serif", size = 9),
    axis.text.x = element_blank(),
    strip.background = element_blank(),
    strip.placement = "outside"
  ) +
  scale_fill_manual(values = c('#377EB8','#4DAF4A','#984EA3','#FF7F00','#FFFF33'),
                    #labels = c("Nutrient-Biomass Integration", "Limited Environmental Intervent
ion","Moderate Multifunctional Outcomes","Water-Quality-Biomass Integration","Prairie-Habitat-Nu
trient Integration")
                    )
```

